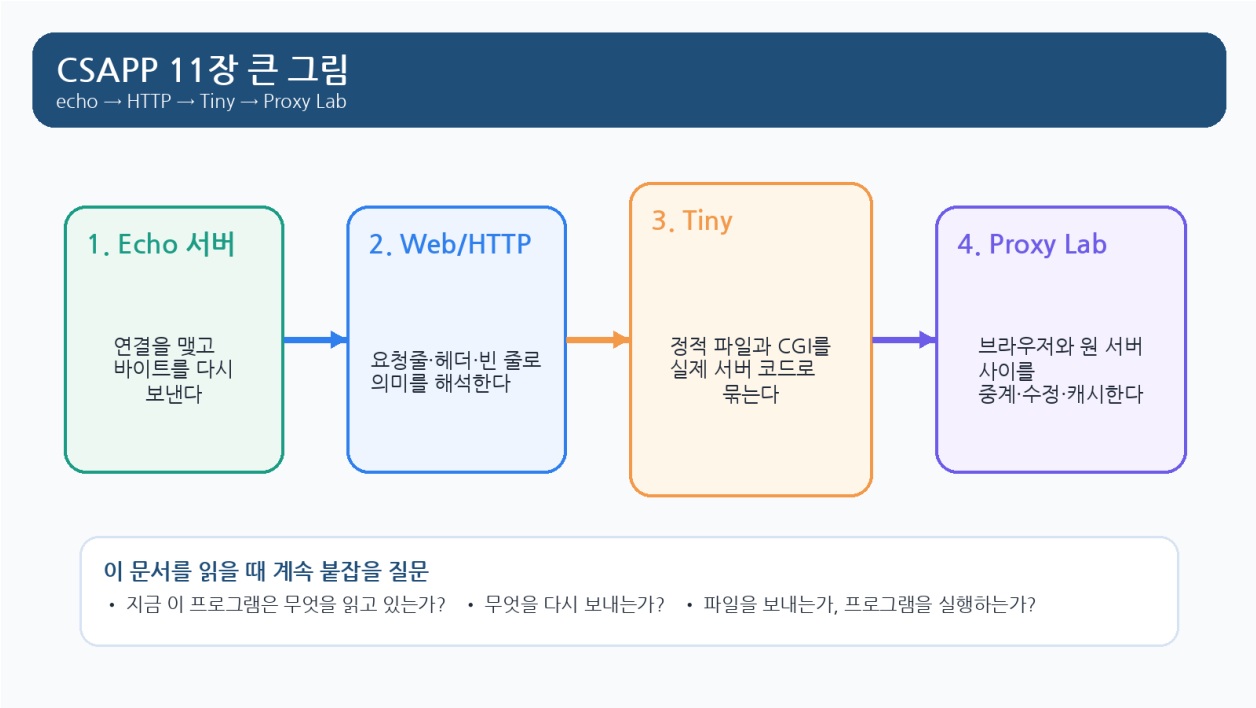
CSAPP 11장 입문 지도

웹 서버를 처음 배우는 사람을 위한 큰 그림 - 11.5 Web Servers 중심 - 11.6 Tiny/Proxy Lab 연결

항목	내용
대상	웹/네트워크를 거의 처음 배우는 학습자, C 문법만 조금 익숙한 수준
핵심 초점	CSAPP 11.5 Web Servers를 가장 깊게 이해하고, 11.6 Tiny와 Proxy Lab으로 자연스럽게 이어지기
설명 방식	수학보다 동작 흐름 중심, 어려운 용어는 처음 등장할 때 바로 쉬운 말로 정의
문서 성격	원문을 읽기 전에 보는 큰 그림 지도 초안



학습 지도의 핵심: echo로 연결을 이해하고, HTTP로 의미를 붙이고, Tiny로 통합한 뒤, Proxy Lab에서 역할을 확장한다.

이 문서의 약속

웹 서버는 특별한 마법이 아니라 **소켓(socket, 네트워크 통신용 연결 끝점)**으로 바이트를 읽고 쓰는 C 프로그램이라는 관점에서 설명합니다.

CSAPP 원문은 Chapter 11의 구조를 따르되, 특히 **11.5 Web Servers**를 가장 자세히 풀고, **11.6 Tiny Web Server**와 **Proxy Lab** 맥락을 계속 연결합니다.

이 초안은 어떻게 읽으면 좋은가

처음부터 모든 세부 코드를 따라가기보다, **지금 이 프로그램이 무엇을 읽고 무엇을 보내는가?**만 계속 붙잡으면 Chapter 11이 훨씬 덜 어렵습니다.

먼저 잡을 것

- 클라이언트(client, 요청을 보내는 쪽)와 서버(server, 응답을 돌려주는 쪽)의 역할 구분
- 한 연결 안에서 요청과 응답이 어떤 순서로 오가는지
- Tiny가 정적 파일과 동적 CGI를 어떻게 갈라서 처리하는지

읽는 순서 추천

- 준비 체크리스트 → 11장 큰 그림 → 11.1~11.4 압축 예열
- 그다음 11.5.1~11.5.4를 천천히
- 마지막에 11.6 Tiny 코드와 Proxy Lab 연결을 보기

지금은 덜 신경 써도 되는 것

- TCP 내부 재전송 알고리즘의 세부 수식
- 현대 브라우저의 모든 헤더와 캐시 정책 세부 규칙
- DNS, TLS, HTTP/2, HTTP/3의 깊은 구현 세부

끝까지 붙잡아야 하는 질문

- 이 바이트는 요청인가, 응답인가?
- 이 URI는 파일을 가리키는가, 프로그램 실행을 가리키는가?
- 지금 열려 있는 파일 디스크립터(fd, 정수 번호로 식별하는 I/O 핸들)는 누구와 연결되어 있는가?

불안 줄이기 메모

브라우저가 수십 개의 헤더를 보낸다고 해서 초반에 모두 이해해야 하는 것은 아닙니다. Tiny가 실제로 중요하게 보는 것은 **요청줄(request line)**과 헤더 끝을 알리는 **빈 줄**, 그리고 정적/동적 분기입니다.

Proxy Lab이 어렵게 느껴지는 이유도 새로운 마법이 생겨서가 아니라, **한 프로그램 안에서 서버 역할과 클라이언트 역할을 동시에 해야 하기 때문**입니다.

목차 및 분량 배치표

아래 쪽수는 이 PDF의 실제 시작 쪽과 실제 분량을 기준으로 계산합니다. 읽기 전에 어느 부분이 긴지 먼저 파악하도록 구성했습니다.

구성	핵심 내용	시작 쪽	배정/실제
이 초안은 어떻게 읽으면 좋은가	읽는 순서, 불안 줄이기, 핵심 질문	2	1 / 1쪽
읽기 전 준비 체크리스트	기초 C/터미널/localhost 감각 확인	5	1 / 1쪽
1. CSAPP 11장 전체를 한 장면으로 보기	echo → HTTP → Tiny → Proxy 큰 그림	6	3 / 2쪽
2. 소켓, 포트, listenfd/connfd를 먼저 몸에 익히기	11.1~11.4 압축 예열	8	3 / 3쪽
3. 웹이란 무엇이고, URL은 무엇을 가리키는가	11.5.1 Web Basics	11	3 / 2쪽
4. 정적 콘텐츠와 동적 콘텐츠는 어떻게 다를까	11.5.2 Web Content	13	3 / 3쪽
5. HTTP 요청(request)은 정확히 어떤 모양인가	11.5.3 요청 메시지	16	4 / 3쪽
6. HTTP 응답(response)과 디버깅 도구를 읽는 눈 만들기	11.5.3 응답 메시지	19	4 / 4쪽

분량 해석 팁

분량이 긴 부분은 대부분 **11.5.3 HTTP 트랜잭션**, **11.5.4 CGI**, 그리고 **11.6 Tiny 코드 해설**입니다. 초보자가 실제로 막히는 지점이 여기에 몰려 있기 때문입니다.

이 초안은 원문 대신 완전한 대체물이 아니라, **원문을 읽을 때 길을 잃지 않게 해 주는 지도** 역할을 하도록 설계했습니다.

목차 및 분량 배치표 (계속)

구성	핵심 내용	시작 쪽	배정/실제
7. CGI: 서버가 다른 프로그램을 실행해 결과를 보내는 법	11.5.4 Serving Dynamic Content	23	4 / 4쪽
8. Tiny는 앞의 개념들을 어떻게 한 파일에 묶는가	11.6 Tiny 전체 흐름	27	4 / 4쪽
9. read_requesthdrs, parse_uri, serve_static를 연결해서 읽기	11.6 Tiny 정적 경로	31	4 / 4쪽
10. serve_dynamic과 adder 예제로 CGI 전체 흐름 끝내기	11.6 Tiny 동적 경로	35	4 / 4쪽
11. 프록시는 왜 Tiny 다음 단계로 배우기 좋은가	Proxy Lab 큰 그림과 HTTP 재구성	39	4 / 4쪽
12. echo, Tiny, Proxy를 어떤 순서로 테스트하면 좋은가	실습/테스트 체크리스트	43	3 / 3쪽
13. 멈춤, 404, 403, 깨진 응답을 보면 어디부터 볼까	공통 함정과 디버깅 순서	46	3 / 3쪽
14. Glossary + 이제 원문 Chapter 11을 어떻게 읽을까	용어사전 + 원문 읽기 로드맵	49	3 / 3쪽

원문과의 연결 기준

- 11.5.1 Web Basics
- 11.5.2 Web Content
- 11.5.3 HTTP Transactions
- 11.5.4 Serving Dynamic Content
- 11.6 Tiny Web Server

이 문서에서 의도적으로 덧붙인 것

- 초보자용 용어 재정의
- 실습/테스트 체크리스트
- Proxy Lab로 이어지는 사고법
- 공통 함정과 디버깅 순서

Before reading CSAPP Chapter 11 - 준비 체크리스트

모든 항목에 체크가 되지 않아도 괜찮습니다. 6개 이상이면 꽤 편안하게, 3~5개면 읽으면서 보강하면서, 0~2개면 echo 실습부터 시작하면 됩니다.

☐ if, while, 함수 호출, 배열, 문자열 같은 기본 C 문법을 읽을 수 있다	☐ 한 프로그램이 다른 프로그램과 데이터를 주고받을 수 있다는 정도의 상식을 안다
☐ 포인터를 완벽히는 몰라도 char * 가 문자열 주소를 가리킨다는 정도는 안다	☐ 브라우저 주소창의 URL에서 호스트/경로 정도는 구분할 수 있다
☐ 터미널에서 make, ./a.out, ./tiny 8000 같은 명령을 실행할 수 있다	☐ 권한(permission) 때문에 실행이 실패할 수 있다는 것을 안다
☐ localhost가 내 컴퓨터 자신을 뜻한다는 것을 안다	☐ curl이나 nc 같은 도구를 처음 봐도 따라 입력할 수 있다
☐ 프로그램이 파일을 열고 읽고 쓴다는 감각이 있다	☐ 응답이 오지 않으면 내가 요청 형식을 잘못 보냈나를 의심할 수 있다
☐ 파일 디스크립터(fd)가 정수 번호라는 사실을 본 적이 있다	☐ 코드가 길어도 입력 → 처리 → 출력 흐름으로 잘라 보려는 습관이 있다

최소 준비물

- C 함수 호출 흐름 읽기
- 문자열과 배열 감각
- 터미널에서 프로그램 실행하기
- localhost/port 정도의 개념

읽기 전 자신에게 물어볼 질문

- 요청과 응답은 둘 다 결국 바이트인가?
- 서버는 파일을 보내기도 하고 프로그램을 실행하기도 하는가?
- 브라우저가 보내는 말과 서버가 돌려주는 말의 형식은 어디에 적혀 있는가?

CHAPTER 1 - 큰 그림 먼저 잡기

1. CSAPP 11장 전체를 한 장면으로 보기

CSAPP Chapter 11 전체 흐름을 먼저 묶어 보는 오리엔테이션

이 장의 학습 목표

- echo 서버, Web/HTTP, Tiny, Proxy Lab이 어떻게 이어지는지 말할 수 있다
- 웹 서버 학습의 핵심 질문이 무엇을 읽고 무엇을 보내는가라는 점을 이해한다
- 11.5가 왜 11장 전체에서 중심 허리 역할을 하는지 설명할 수 있다

시각 자료 가이드

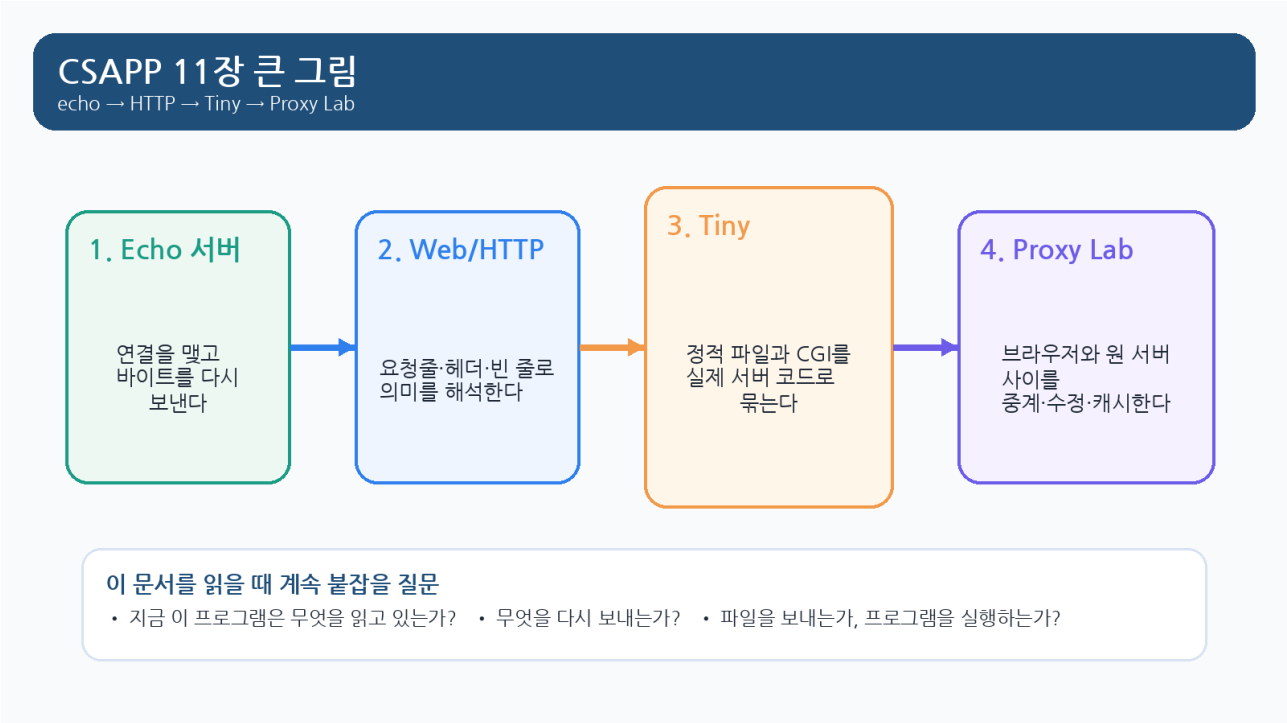
- 학습 지도 다이어그램: echo → HTTP → Tiny → Proxy 흐름을 한눈에 보여 주기 위해
- 단계별 비교표: 각 단계가 새로 떠안는 책임을 비교하기 위해

왜 먼저 알아두면 좋은가

Chapter 11은 네트워크 함수 이름을 외우는 장이 아니라, **C 프로그램이 웹이라는 약속 위에서 어떻게 동작하는지** 감각을 잡는 장입니다. 먼저 큰 그림을 잡아 두면 뒤의 코드가 훨씬 덜 낯섭니다.

CSAPP 11장은 네트워크 프로그래밍을 추상적으로만 설명하는 장이 아니라, **웹 서버(web server, 브라우저의 요청을 받아 HTTP 응답을 보내는 프로그램)**를 손으로 만들며 네트워크 I/O를 이해하게 만드는 장입니다. 그래서 한 번에 모든 것을 배우기보다, **연결 → HTTP 형식 → 파일/프로그램 응답 → 중간자 확장** 순서로 올라갑니다.

초보자 눈에는 echo 서버와 Tiny가 전혀 다른 프로그램처럼 보일 수 있습니다. 하지만 큰 틀은 같습니다. 둘 다 **연결을 받아들인 뒤(accept), 클라이언트가 보낸 바이트를 읽고, 무언가를 다시 쓰고, 연결을 닫습니다**. 차이는 읽은 바이트를 얼마나 깊게 해석하느냐에 있습니다.



echo는 연결과 왕복을 익히는 출발점이고, 11.5는 HTTP 의미를 붙이는 구간이며, 11.6 Tiny는 그 둘을 묶어 실제 웹 서버로 만든다.

단계	프로그램이 새로 떠안는 책임	초보자가 붙잡을 질문
Echo 서버	소켓 연결을 받아 텍스트를 다시 돌려준다	연결은 어떻게 열리고 닫히는가?
11.5 Web/HTTP	요청줄, 헤더, 본문이 무엇인지 해석한다	이 바이트는 무슨 뜻인가?
11.6 Tiny	URI를 보고 정적 파일인지 CGI 실행인지 갈라서 응답한다	파일을 보내는가, 프로그램을 실행하는가?
Proxy Lab	클라이언트와 원 서버 사이에서 중계, 수정, 캐시한다	지금 나는 어느 쪽 입장으로 말하고 있는가?

초보자 관찰 포인트

11.1~11.4는 **소켓 API(socket API, 프로그램이 네트워크 연결을 파일처럼 다루게 해 주는 함수 묶음)**를 익히는 준비 운동입니다.

11.5는 그 위에 **웹의 의미**를 얹습니다. 요청줄, 헤더, 정적/동적 콘텐츠 구분 같은 개념이 여기서 들어옵니다.

11.6 Tiny는 앞에서 배운 조각이 여기서 만나는구나 하는 순간을 만들어 줍니다.

Proxy Lab은 Tiny보다 완전히 새로운 세계가 아니라, 같은 사고법을 더 넓힌 확장판입니다. 받는 쪽에서는 서버처럼, 보내는 쪽에서는 클라이언트처럼 행동하면 됩니다.

마무리 체크

3-line summary

- Chapter 11의 핵심은 복잡한 네트워크 이론보다 바이트를 어떤 규칙으로 해석하느냐에 있다.
- 11.5는 HTTP와 웹 콘텐츠 개념을 붙여 주는 허리이고, 11.6 Tiny는 그 개념을 코드로 묶어 낸다.
- Proxy Lab은 Tiny와 단절된 주제가 아니라, 같은 흐름에 중계와 캐시 역할을 더한 단계다.

2 self-check questions

1) Echo 서버와 Tiny의 공통 뼈대는 무엇인가?
2) 왜 11.5를 이해하지 못하면 11.6 코드를 읽어도 길을 잃기 쉬운가?

초보자 메모

이 절을 “무엇을 읽고, 어떻게 처리하고, 무엇을 보내는가?”라는 한 문장으로 다시 말해 보면 이해가 훨씬 단단해집니다. 가능하면 브라우저 화면만 보지 말고, curl -v 또는 nc 로 이 절의 예시를 한 번 직접 확인해 보세요.

내 한 줄 메모

CHAPTER 2 - 11.1~11.4 압축 예열

2. 소켓, 포트, listenfd/connfd를 먼저 몸에 익히기

CSAPP 11.1~11.4를 초보자 시선에서 압축 정리

이 장의 학습 목표

- 클라이언트와 서버, 호스트, 포트, 소켓의 역할을 말할 수 있다
- 리스닝 소켓(listenfd)과 연결 소켓(connfd)의 차이를 설명할 수 있다
- 반복적 서버(iterative server)의 기본 루프를 읽을 수 있다

시각 자료 가이드

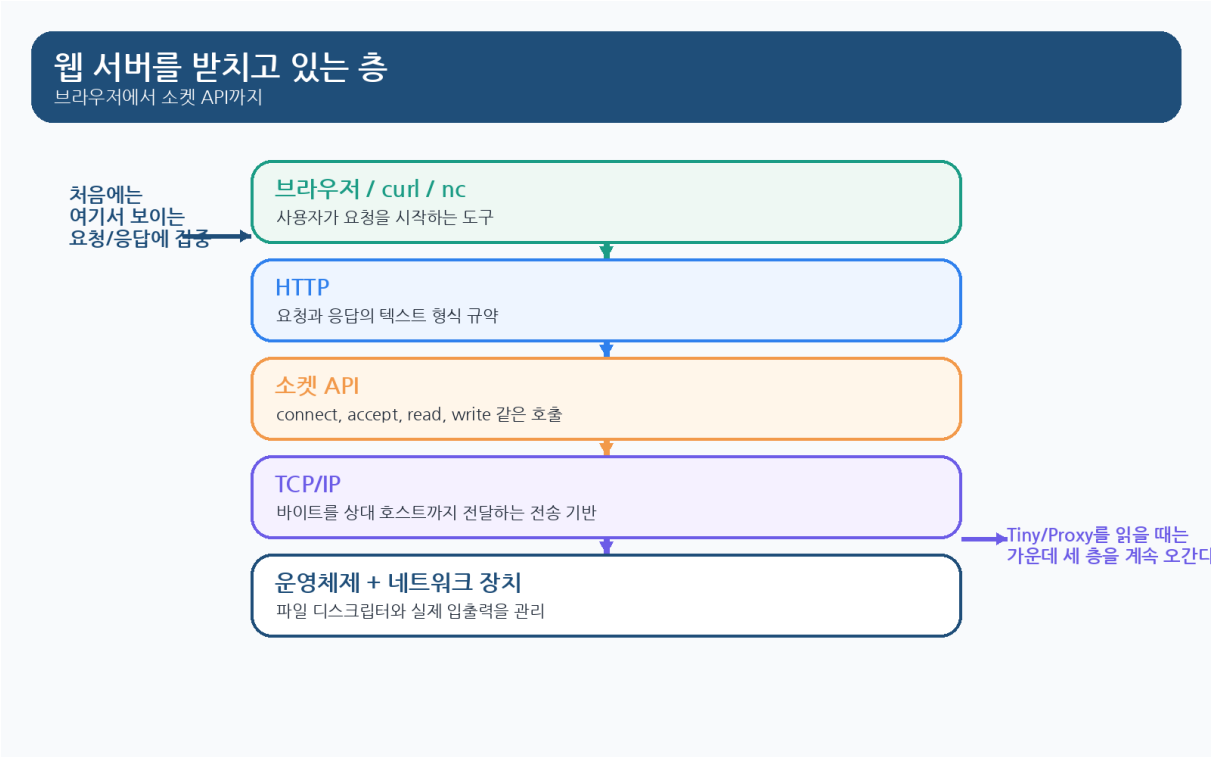
- 층 그림: 브라우저에서 소켓 API까지 역할을 구분하기 위해
- listenfd vs connfd 비교표: 두 fd를 헷갈리지 않게 하기 위해

왜 먼저 알아두면 좋은가

11.5의 HTTP 설명은 이미 연결이 된 뒤를 전제로 합니다. 그래서 먼저 **연결을 만드는 도구**인 소켓 관련 감각을 짧게 잡아 두는 것이 좋습니다.

먼저 **클라이언트(client, 요청을 시작하는 쪽)**와 **서버(server, 요청을 받아 응답하는 쪽)**를 구분해야 합니다. 브라우저가 웹 서버에 접근할 때 브라우저는 클라이언트이고, Tiny는 서버입니다. Proxy는 상황에 따라 둘 다 됩니다.

호스트(host, 네트워크에 연결된 컴퓨터)를 찾으려면 주소가 필요하고, 한 호스트 안에서 어떤 프로그램에게 전달할지 고르려면 **포트(port, 프로그램을 구분하는 번호)**가 필요합니다. 예를 들어 `localhost:8000` 은 내 컴퓨터의 8000번 포트에서 기다리는 프로그램이라는 뜻입니다.



초반에는 모든 층을 완벽히 외우기보다, HTTP가 소켓 API 위에서 동작한다는 층 분리 감각만 잡아도 충분하다.

listenfd와 connfd를 꼭 구분하기

리스닝 소켓(listening socket)은 누가 오면 받겠다고 문 앞에서 기다리는 역할입니다. 반면 **연결 소켓(connected socket)**은 실제로 한 클라이언트와 대화할 때 쓰는 별도의 fd입니다. 같은 서버 안에 둘이 동시에 존재합니다.

구분	무엇을 위한 fd인가?	초보자 이미지
listenfd	특정 포트에서 새 연결을 기다림	가게 입구 문
connfd	받아들인 한 손님과 실제로 대화함	카운터에 앉은 손님과의 대화 창구

반복적 서버의 가장 작은 뼈대

```
listenfd = Open_listenfd(port);
while (1) {
    connfd = Accept(listenfd, NULL, NULL);
    doit(connfd);
    Close(connfd);
}
```

Tiny도 결국 이 반복 루프 위에 HTTP 해석과 파일/CGI 처리가 붙은 형태다.

이 코드는 아주 중요합니다. 여기서 서버는 **한 번에 하나의 연결**만 처리합니다. 그래서 이런 서버를 **반복적(iterative) 서버**라고 부릅니다. Tiny는 학습용 예제이기 때문에 이 단순한 구조를 유지합니다.

연결이 끝났다는 뜻의 **EOF(end of file, 더 읽을 바이트가 없다는 상태)**도 디스크 파일과 인터넷 연결에서 약간 다르게 느껴질 수 있습니다. 네트워크에서는 보통 상대 프로세스가 자기 쪽 연결을 닫았을 때 read 가 0을 돌려주며 EOF를 감지합니다.

자주 막히는 지점

listenfd 하나로 모든 대화를 하는 것처럼 생각하면 금방 헷갈립니다. 새 연결을 받는 fd와, 이미 받은 상대와 대화하는 fd를 분리해서 보세요.

포트가 이미 사용 중이면 프로그램이 시작조차 안 될 수 있습니다. 이때는 다른 포트를 쓰거나, 이전 프로세스가 살아 있는지 확인해야 합니다.

connfd 를 닫지 않으면 파일 디스크립터가 쌓여 이상 동작으로 이어질 수 있습니다.

마무리 체크

3-line summary

- 소켓은 프로그램이 네트워크 연결을 파일처럼 다루게 해 주는 통로다.
- listenfd는 새 손님을 기다리고, connfd는 실제 손님과 대화한다.
- Tiny의 큰 반복 루프는 Accept → doit → Close 구조를 갖는 반복적 서버다.

2 self-check questions

1) 왜 서버 안에 fd가 두 종류로 나뉘어 있는가?
2) 반복적 서버가 단순한 대신 잃는 것은 무엇일까?

초보자 메모

이 절을 “무엇을 읽고, 어떻게 처리하고, 무엇을 보내는가?”라는 한 문장으로 다시 말해 보면 이해가 훨씬 단단해집니다. 가능하면 브라우저 화면만 보지 말고, curl -v 또는 nc 로 이 절의 예시를 한 번 직접 확인해 보세요.

내 한 줄 메모

CHAPTER 3 - 11.5.1 Web Basics

3. 웹이란 무엇이고, URL은 무엇을 가리키는가

CSAPP 11.5.1 Web Basics에 대응

이 장의 학습 목표

- 브라우저와 웹 서버가 어떤 역할을 나눠 가지는지 설명할 수 있다
- URL/URI의 앞부분과 뒷부분이 각각 어떤 결정을 돕는지 이해한다
- 웹이 문서의 집합이면서 동시에 리소스(resource, 제공 가능한 대상)의 집합이라는 감각을 잡는다

시각 자료 가이드

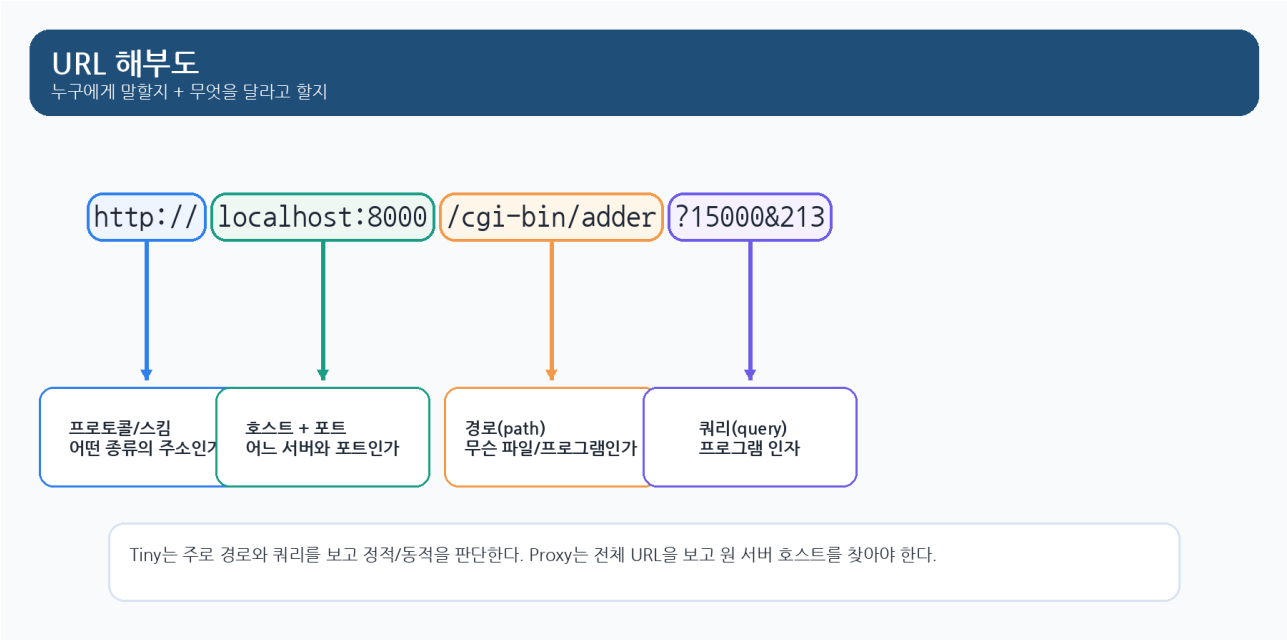
- URL 해부 그림: 호스트, 포트, 경로, 인자를 분해해 보여 주기 위해
- 개념 박스: 클라이언트가 보는 부분과 서버가 보는 부분을 나눠 보기 위해

왜 먼저 알아두면 좋은가

11.5.1은 웹은 그냥 브라우저로 보는 화면이라는 막연한 감각을, **주소·요청·응답**이라는 프로그래머 관점으로 바꾸는 구간입니다.

웹에서는 브라우저가 **URL(URL, 자원을 찾기 위한 주소 문자열)**을 보고 요청을 만들고, 서버는 그 주소를 해석해 무엇을 보낼지 결정합니다. 이때 서버가 보내는 것은 꼭 HTML 파일만이 아닙니다. 이미지, 동영상, CSS, 실행 결과처럼 무엇이든 응답으로 보낼 수 있는 대상이라면 다 **리소스(resource)**라고 볼 수 있습니다.

초보자에게는 URL이 단순한 문자열처럼 보이지만, 사실은 어느 서버에게 말할지와 무엇을 달라고 할지가 함께 들어 있습니다. 앞부분은 대체로 서버를 찾는 데 쓰이고, 뒷부분은 서버가 어떤 콘텐츠를 선택할지 결정하는 데 쓰입니다.



Tiny와 Proxy 관점에서 보면, Proxy는 전체 URL에서 원 서버를 찾아야 하고, Tiny는 주로 경로(path)와 인자(query)를 로컬 규칙으로 해석한다.

URL과 URI를 초보자 관점에서 구분하기

URI(Uniform Resource Identifier)는 자원을 식별하는 문자열 전체를 가리키는 넓은 말입니다. 실습에서는 URL과 URI를 엄격히 구분하지 않아도 괜찮습니다. 중요한 것은 문자열 안에 **호스트(host)**, **포트(port)**, **경로(path)**, **쿼리 문자열(query string)** 같은 조각이 들어 있다는 사실입니다.

예를 들어 `http://localhost:8000/cgi-bin/adder?1&2` 에서 `localhost:8000` 은 누구에게 말할지, `/cgi-bin/adder?1&2` 는 무엇을 달라고 할지를 나타냅니다. Tiny는 이 뒷부분을 보고 정적 파일인지 CGI 실행인지 판단합니다.

누가 주로 본다고 생각하면 편한가	관심 조각	왜 필요한가
클라이언트/Proxy	http://host:port	어느 호스트와 포트로 연결할지 결정하기 위해
서버(Tiny)	/path 와 ?query	어느 파일을 읽거나 어느 프로그램을 실행할지 결정하기 위해

브라우저는 내부적으로 무엇을 하는가

사용자가 링크를 클릭하면 브라우저는 URL을 보고 요청을 만듭니다. 그런 다음 서버와 연결한 뒤, **HTTP(웹에서 요청과 응답 형식을 정한 규약)**에 맞는 텍스트를 보내고, 되돌아온 응답을 화면으로 렌더링합니다. 즉 브라우저도 결국은 받은 바이트를 해석하는 프로그램입니다.

CSAPP 원문이 강조하는 포인트 하나는, 서버가 URL의 뒷부분을 해석하는 규칙은 절대적인 자연법칙이 아니라 **서버가 정한 로컬 규칙**이라는 점입니다. Tiny가 `cgi-bin` 을 특별 취급하는 것도 그 규칙의 한 예입니다.

현실과 원문의 거리

현실의 브라우저와 서버는 HTTP/2, HTTP/3 같은 더 현대적인 버전도 사용합니다. 하지만 CSAPP 11장은 HTTP/1.0/1.1 수준의 단순한 예제로 **핵심 구조**를 보여 줍니다. 초보자는 이 단순한 구조부터 확실히 잡는 편이 훨씬 좋습니다.

마무리 체크

3-line summary

- 웹의 핵심은 화면 그 자체보다, 주소를 통해 자원을 요청하고 응답으로 받는 구조에 있다.
- URL은 누구에게 말할지와 무엇을 달라고 할지를 함께 담고 있다.
- 브라우저도 서버도 결국은 규약에 맞춰 바이트를 읽고 쓰는 프로그램이다.

2 self-check questions

1) Proxy와 Tiny는 URL의 어느 부분에 특히 관심이 많은가?
2) 왜 CSAPP는 최신 HTTP 기능보다 HTTP/1.0 스타일 예제로 시작할까?

초보자 메모

이 절을 “무엇을 읽고, 어떻게 처리하고, 무엇을 보내는가?”라는 한 문장으로 다시 말해 보면 이해가 훨씬 단단해집니다. 가능하면 브라우저 화면만 보지 말고, curl -v 또는 nc 로 이 절의 예시를 한 번 직접 확인해 보세요.

내 한 줄 메모

CHAPTER 4 · 11.5.2 Web Content

4. 정적 콘텐츠와 동적 콘텐츠는 어떻게 다를까

CSAPP 11.5.2 Web Content에 대응

이 장의 학습 목표

- 정적 콘텐츠(static content)와 동적 콘텐츠(dynamic content)의 차이를 설명할 수 있다
- 둘이 만들어지는 방식은 달라도 결국 HTTP 응답으로 수렴한다는 점을 이해한다
- Tiny가 URI를 보고 정적/동적 경로를 나누는 이유를 말할 수 있다

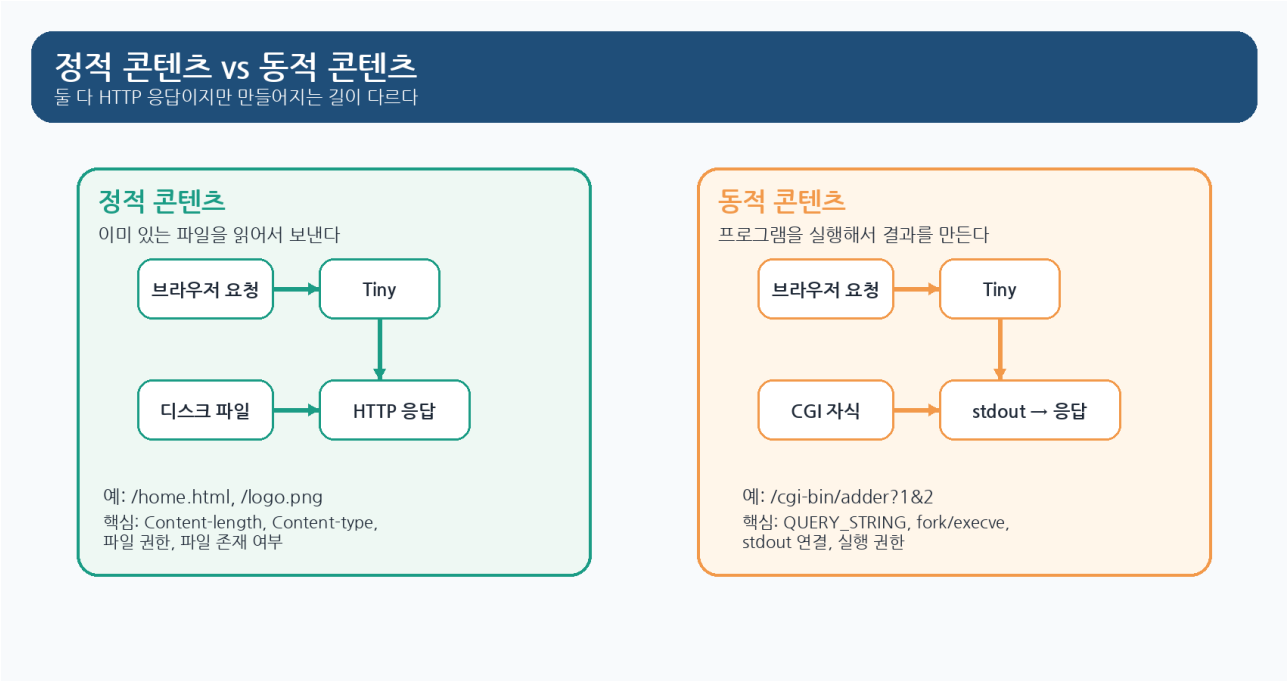
시각 자료 가이드

- 정적 vs 동적 그림: 두 경로의 차이를 한 장면에서 비교하기 위해
- MIME/규칙 표: 파일 종류와 브라우저 해석의 연결을 보여 주기 위해

왜 먼저 알아두면 좋은가

11.5.2는 Tiny 코드를 읽을 때 가장 중요한 기준 하나를 줍니다. 바로 **이 요청은 파일을 보내는가, 지금 프로그램을 실행하는가?**라는 기준입니다.

정적 콘텐츠(static content)는 디스크에 이미 있는 파일을 거의 그대로 보내는 경우입니다. 예를 들어 HTML, 이미지, CSS 파일이 여기에 들어갑니다. 반면 **동적 콘텐츠(dynamic content)**는 요청이 들어온 뒤에 프로그램을 실행해서 그때그때 출력을 만드는 경우입니다. Tiny에서는 CGI 프로그램이 이 역할을 합니다.



둘 다 최종 결과는 HTTP 응답이지만, 정적은 파일을 읽고, 동적은 프로그램을 실행해 출력한다는 점이 다르다.

중요한 점은 두 경우 모두 **최종적으로는 HTTP 응답 바이트**가 된다는 것입니다. 브라우저 입장에서는 서버가 파일을 보냈는지, 프로그램 실행 결과를 보냈는지가 보통 드러나지 않습니다. 브라우저는 그저 응답 헤더와 본문을 받아 해석할 뿐입니다.

MIME 타입은 왜 필요한가

MIME 타입(MIME type, 받은 데이터가 어떤 종류인지 알려 주는 문자열)은 브라우저가 본문을 어떻게 다룰지 결정하는 데 도움을 줍니다. 예를 들어 `text/html` 이면 HTML로, `image/png` 이면 PNG 이미지로 해석합니다. Tiny의 `get_filetype` 도 파일 확장자를 보고 이 타입을 정합니다.

MIME 타입	브라우저가 기대하는 것
text/html	HTML 문서로 해석해 화면을 구성
text/plain	서식 없는 일반 텍스트로 표시
image/gif / image/png / image/jpeg	이미지로 렌더링
application/octet-stream	정해진 해석이 없으면 내려받기 대상으로 보기도 함

Tiny의 단순한 분기 규칙

```
if ("cgi-bin" not in uri) {
    // static: 로컬 파일 경로로 해석
} else {
    // dynamic: CGI 프로그램 + 인자 분리
}
```

Tiny는 학습용 예제라서 cgi-bin 문자열 유무로 정적/동적을 아주 단순하게 구분한다.

현실의 웹 서버는 더 복잡한 규칙을 씁니다. 하지만 Tiny는 학습용이므로 **문자열 안에 cgi-bin 이 있으면 동적**이라고 가정합니다. 초보자에게는 이 단순함이 오히려 장점입니다. 코드를 읽을 때 분기가 눈에 잘 들어오기 때문입니다.

자주 오해하는 부분

동적 콘텐츠라고 해서 꼭 POST 요청이거나 데이터베이스가 필요한 것은 아닙니다. Tiny의 `/cgi-bin/adder?1&2` 는 아주 단순한 GET 기반 동적 콘텐츠입니다.

정적 콘텐츠와 동적 콘텐츠는 프로토콜이 다른 것이 아니라, **응답이 만들어지는 방식**이 다른 것입니다.

서버가 URL의 뒷부분을 어떻게 파일 경로나 프로그램 경로로 해석할지는 서버 로컬 규칙에 달려 있습니다.

마무리 체크

3-line summary

- 정적 콘텐츠는 보통 디스크 파일을 읽어 보내고, 동적 콘텐츠는 프로그램을 실행해 결과를 만든다.
- 둘 다 결국은 HTTP 응답 헤더와 본문으로 브라우저에 전달된다.
- Tiny는 교육용 예제로 cgi-bin 문자열을 기준으로 정적/동적을 아주 단순하게 나눈다.

2 self-check questions

1) 브라우저는 정적 응답과 동적 응답을 어떤 수준에서 구분할까?
2) Tiny가 단순한 분기 규칙을 쓰는 것이 왜 학습에는 유리할까?

초보자 메모

이 절을 “무엇을 읽고, 어떻게 처리하고, 무엇을 보내는가?”라는 한 문장으로 다시 말해 보면 이해가 훨씬 단단해집니다. 가능하면 브라우저 화면만 보지 말고, curl -v 또는 nc 로 이 절의 예시를 한 번 직접 확인해 보세요.

내 한 줄 메모

CHAPTER 5 - 11.5.3 HTTP Transactions I

5. HTTP 요청(request)은 정확히 어떤 모양인가

CSAPP 11.5.3 HTTP Transactions의 요청 쪽을 중심으로

이 장의 학습 목표

- 요청줄(request line)의 세 조각(method, URI, version)을 읽을 수 있다
- 헤더(header)와 빈 줄(blank line)의 의미를 설명할 수 있다
- GET과 POST가 어디에 인자를 싣는지 큰 차원에서 이해한다

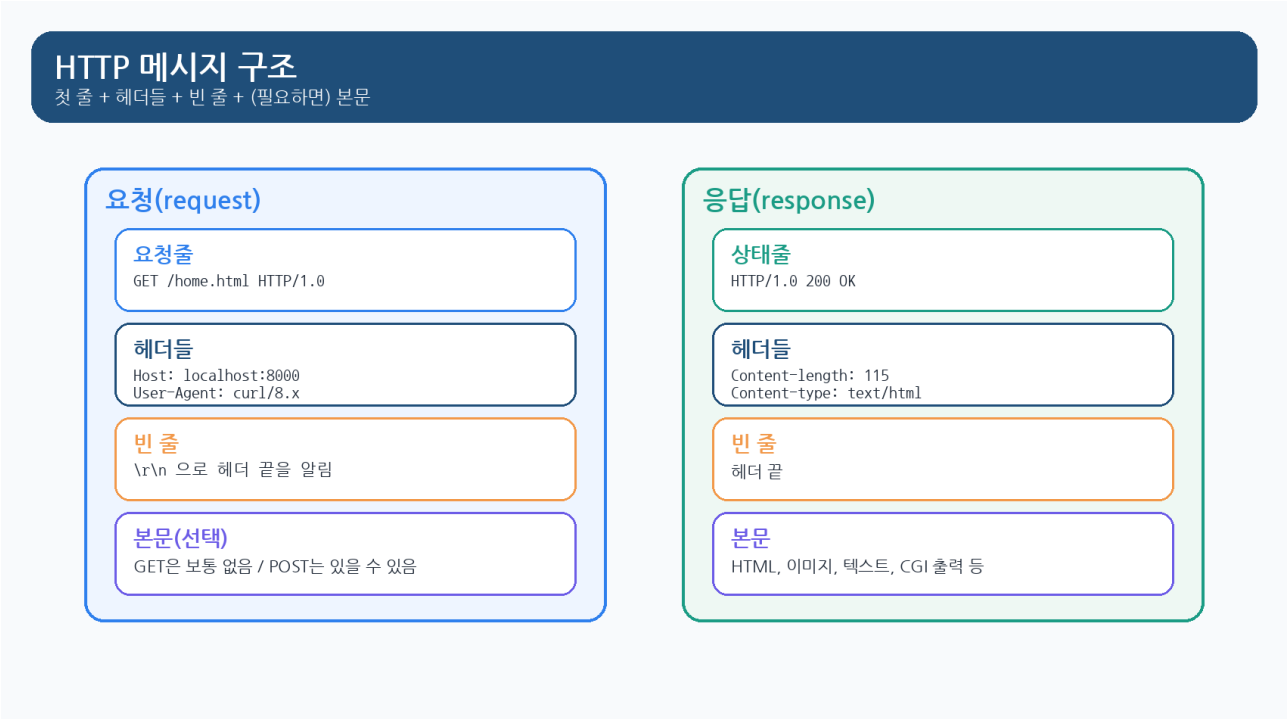
시각 자료 가이드

- HTTP 메시지 구조 그림: 요청과 응답의 공통 골격을 한눈에 보여 주기 위해
- 짧은 raw 요청 예시: 빈 줄의 중요성을 직접 느끼게 하기 위해

왜 먼저 알아두면 좋은가

11.5.3은 웹은 그냥 브라우저가 알아서 하는 것이라는 느낌을 깨고, 요청이 사실은 **사람이 읽을 수 있는 텍스트 줄**이라는 점을 보여 줍니다.

HTTP의 한 번의 **트랜잭션(transaction, 요청 한 번과 그에 대한 응답 한 번의 왕복)**은 보통 요청과 응답 쌍으로 이루어집니다. 요청의 첫 줄이 바로 **요청줄(request line)**입니다. 여기에는 **메서드(method, 무엇을 하려는지)**, **URI**, **버전(version)**이 들어갑니다.



요청과 응답은 모두 첫 줄 + 헤더들 + 빈 줄 + (필요하면) 본문 구조를 가진다.

```
GET /home.html HTTP/1.0
Host: localhost:8000
User-Agent: curl/8.x
```

헤더가 끝났다는 사실을 알려 주는 마지막 빈 줄이 매우 중요하다.

첫 줄의 `GET` 은 이 자원을 가져오라는 뜻입니다. 뒤의 `/home.html` 은 무엇을 달라는지, `HTTP/1.0` 은 어떤 버전 규칙으로 말하겠다는지 알려 줍니다.

헤더(header)는 부가 정보 줄입니다. `Host` 는 어떤 호스트를 대상으로 하는지, `User-Agent` 는 누가 요청을 보내는지 등을 알려 줍니다. Tiny는 많은 헤더를 무시해도 되지만, **헤더를 모두 읽어서 마지막 빈 줄까지 지나가야** 다음 단계로 안전하게 갈 수 있습니다.

GET과 POST를 초보자 관점에서만 구분하기

항목	GET	POST
주 용도	조회/가져오기 성격으로 자주 사용	데이터 전송/제출 성격으로 자주 사용
인자 위치	URL 뒤 쿼리 문자열에 붙는 경우가 많음	요청 본문(body)에 실리는 경우가 많음
Tiny 지원	지원	기본 Tiny는 지원하지 않음

브라우저가 프록시에게 보낼 때는 **절대 URI(absolute URI)** 형태인 `GET http://host:port/path HTTP/1.1` 처럼 보낼 수 있습니다. 반면 원 서버가 기대하는 것은 보통 `GET /path HTTP/1.0` 처럼 상대 경로에 가까운 형태입니다. 이 차이를 이해하면 Proxy Lab의 헤더 재작성 이유가 훨씬 선명해집니다.

작게 but 중요하게

HTTP 줄 끝은 보통 `\r\n` 으로 끝난다고 생각하면 좋습니다. nc 나 printf 로 손수 요청을 보낼 때 이 줄바꿈을 빠뜨리면 서버와 도구가 서로 기다리며 멈춘 것처럼 보일 수 있습니다.

브라우저는 대개 HTTP/1.1 스타일 요청을 보냅니다. Proxy Lab은 이런 요청을 받아, 과제 규칙에 맞게 HTTP/1.0 형태로 다시 정리해 원 서버로 보내는 연습을 하게 됩니다.

마무리 체크

3-line summary

- HTTP 요청의 핵심은 요청줄과 헤더들, 그리고 헤더 끝을 나타내는 빈 줄이다.
- GET은 주로 URL 쪽에 인자를 싣고, POST는 본문 쪽에 싣는 경우가 많다.
- Tiny가 헤더 내용을 다 쓰지 않더라도 빈 줄까지 읽어야 하는 이유가 있다.

2 self-check questions

1) 왜 서버는 필요 없어 보이는 헤더도 일단 읽고 지나가야 할까?
2) 프록시에게 보내는 요청과 원 서버에게 보내는 요청의 URI 모양은 왜 달라질까?

초보자 메모

이 절을 “무엇을 읽고, 어떻게 처리하고, 무엇을 보내는가?”라는 한 문장으로 다시 말해 보면 이해가 훨씬 단단해집니다. 가능하면 브라우저 화면만 보지 말고, curl -v 또는 nc 로 이 절의 예시를 한 번 직접 확인해 보세요.

내 한 줄 메모

보강 노트 - 요청 메시지를 한 줄씩 손으로 읽기

브라우저는 요청을 자동으로 만들어 주기 때문에 처음에는 HTTP 요청이 안 보입니다. 그래서 Tiny/Proxy 실습에서는 **직접 한 줄씩 읽고 손으로 보내 보는 연습**이 아주 큰 도움이 됩니다.

특히 Proxy Lab으로 가면 브라우저가 보낸 원래 요청을 받아, 과제 규칙에 맞게 일부 헤더를 정리해서 다시 보내야 합니다. 그래서 **각 줄이 왜 있는지**를 아는 것이 중요합니다.

줄	왜 있는가	Tiny/Proxy에서 보는 포인트
GET /index.html HTTP/1.0	무엇을 어떤 버전으로 달라는지 알려 준다	Tiny는 경로를 보고 정적/동적을 가른다
Host: localhost:8000	어느 호스트를 대상으로 하는지 알려 준다	브라우저는 주로 넣고, Proxy는 원 서버 기준으로 다시 써야 할 때가 있다
User-Agent: ...	누가 요청했는지 알려 준다	Proxy Lab은 지정된 User-Agent 헤더를 쓰도록 요구하는 경우가 많다
Connection: close	이번 응답 후 연결을 닫자고 약속한다	학습용 HTTP/1.0 흐름을 단순하게 유지해 준다
빈 줄	헤더가 여기서 끝났다는 신호다	이 줄이 없으면 서버는 아직 더 읽어야 하는 줄 알고 기다릴 수 있다

```
printf 'GET /cgi-bin/adder?1&2 HTTP/1.0\r\nHost: localhost:8000\r\nConnection: close\r\n\r\n' \
| nc localhost 8000
```

브라우저 없이도 요청을 직접 보내 보면, 헤더와 빈 줄의 역할이 훨씬 또렷해진다.

작은 함정 3개

줄바꿈을 그냥 ₩n 하나로만 보내면 환경에 따라 다르게 보일 수 있습니다. 실습에서는 HTTP 표기대로 `\r\n`을 의식해 두면 좋습니다.

빈 줄을 빼먹으면 서버는 헤더가 아직 끝나지 않았다고 생각할 수 있습니다.

Tiny는 보통 **경로 중심 URI**를 기대하고, Proxy는 브라우저가 보낸 **전체 URL**을 만날 수 있습니다. 같은 GET이라도 보는 문자열 모양이 다를 수 있습니다.

마무리 체크

3-line summary

- HTTP 요청은 거대한 마법이 아니라 사람이 읽을 수 있는 여러 줄의 텍스트다.
- 각 헤더는 부가 정보를 담고 있고, 마지막 빈 줄은 헤더 종료를 알리는 신호다.
- Proxy Lab으로 갈수록 각 줄을 읽고 다시 조립하는 능력이 중요해진다.

2 self-check questions

1) 왜 빈 줄 하나가 없으면 서버가 멈춘 것처럼 보일 수 있을까?
2) Tiny와 Proxy가 같은 GET 요청에서도 서로 다른 모양의 URI를 만날 수 있는 이유는 무엇일까?

초보자 메모

이 절을 “무엇을 읽고, 어떻게 처리하고, 무엇을 보내는가?”라는 한 문장으로 다시 말해 보면 이해가 훨씬 단단해집니다. 가능하면 브라우저 화면만 보지 말고, curl -v 또는 nc 로 이 절의 예시를 한 번 직접 확인해 보세요.

내 한 줄 메모

CHAPTER 6 - 11.5.3 HTTP Transactions II

6. HTTP 응답(response)과 디버깅 도구를 읽는 눈 만들기

CSAPP 11.5.3 HTTP Transactions의 응답 쪽을 중심으로

이 장의 학습 목표

- 상태줄(status line), 상태 코드(status code), 응답 헤더의 역할을 설명할 수 있다
- Content-length와 Content-type이 왜 자주 등장하는지 이해한다
- curl -v, curl -I, nc 같은 도구로 응답을 확인하는 기본 습관을 익힌다

시각 자료 가이드

- 요청-응답 타임라인: 한 연결 안에서 흐름이 시간 순서대로 어떻게 진행되는지 보이기 위해
- 대표 상태 코드 표: 200/403/404/501의 감각을 빨리 익히기 위해

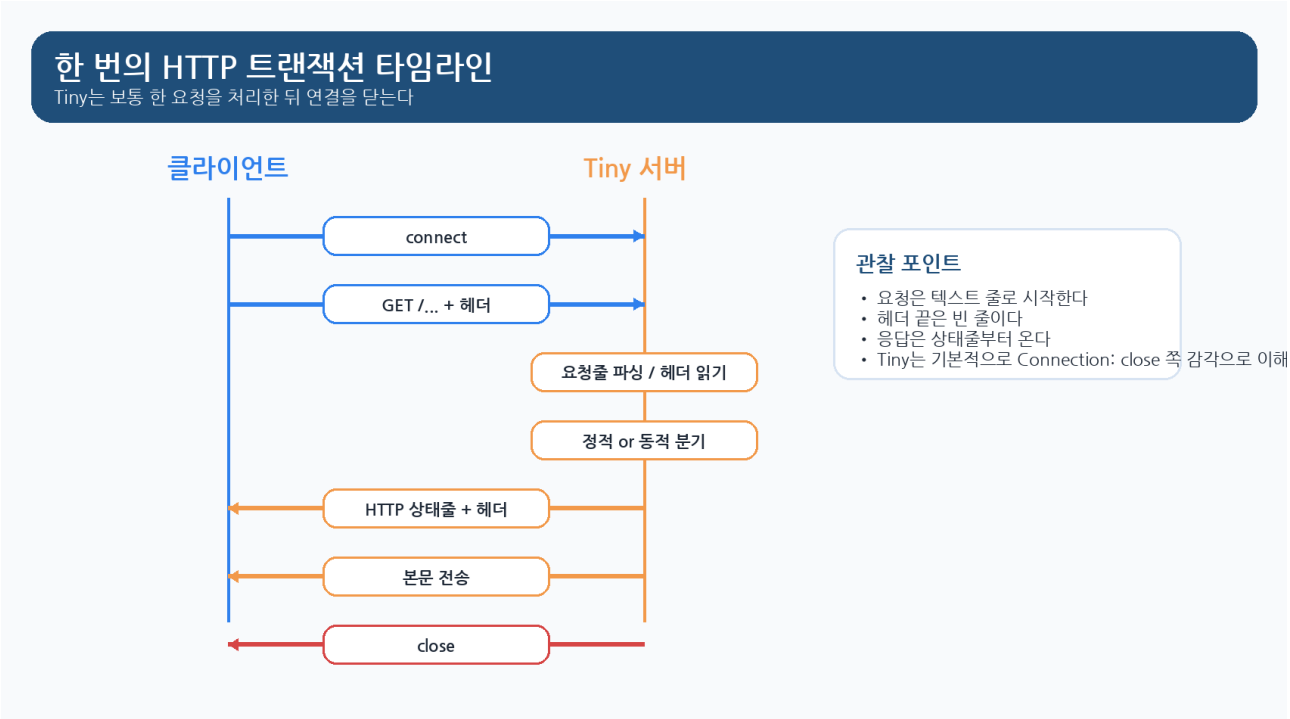
왜 먼저 알아두면 좋은가

응답을 읽을 줄 알면 Tiny의 코드가 훨씬 덜 추상적으로 보입니다. 서버 함수들이 결국 어떤 줄들을 만들고 있는지 눈으로 보이기 시작하기 때문입니다.

응답의 첫 줄은 **상태줄(status line)**입니다. 예를 들어 `HTTP/1.0 200 OK` 는 HTTP/1.0 규칙으로, 처리 결과는 성공(200)이라는 뜻입니다. 그 다음에 오는 응답 헤더들은 본문 길이, 종류, 서버 이름 같은 정보를 알려 줍니다.

코드	이름	초보자 의미
200	OK	요청을 정상 처리했고 본문을 줄 수 있다
301	Moved Permanently	다른 위치로 옮겨졌다는 뜻의 리다이렉트
400	Bad Request	요청 형식이 잘못되었다
403	Forbidden	파일은 있지만 읽거나 실행할 권한이 없다
404	Not Found	요청한 파일/프로그램 경로를 찾지 못했다
501	Not Implemented	서버가 그 메서드(예: POST)를 지원하지 않는다
505	HTTP Version Not Supported	요청한 버전을 지원하지 않는다

`Content-length` 는 본문 길이를 알려 주어, 클라이언트가 어디까지 읽어야 하는지 판단하는 데 도움을 줍니다. `Content-type` 은 브라우저가 본문을 HTML, 이미지, 텍스트 중 무엇으로 볼지 정하는 데 중요합니다.



Tiny는 보통 한 연결에서 한 요청만 처리하고 닫는 HTTP/1.0 스타일 흐름을 보여 준다.

터미널에서 직접 응답 보기

```
# Tiny 실행 후
curl -v http://localhost:8000/

# 헤더만 보고 싶다면
curl -I http://localhost:8000/

# 요청을 손으로 보낸다면
printf "GET / HTTP/1.0\r\n\r\n" | nc localhost 8000
```

브라우저 대신 curl과 nc를 쓰면 서버가 실제로 어떤 줄을 주고받는지 더 선명하게 보인다.

디버깅할 때 브라우저 하나만 쓰면 캐시나 자동 요청 때문에 헷갈릴 수 있습니다. **curl -v** 는 요청과 응답 헤더를 그대로 보여 주기 때문에 Tiny/Proxy 실습에서 매우 유용합니다. **nc(netcat)** 는 아예 요청을 손으로 만들어 볼 수 있어서 빈 줄이나 버전 문자열 문제를 잡는 데 좋습니다.

HTTP/1.1은 한 연결에서 여러 트랜잭션을 이어서 처리하는 **지속 연결(persistent connection)**을 지원할 수 있지만, Tiny는 학습용으로 `Connection: close` 감각에 훨씬 가깝게 이해하면 편합니다. Proxy Lab이 일부러 `Connection: close` 를 붙이는 이유도 여기에 있습니다.

초보자 디버깅 습관

응답이 이상하면 먼저 상태줄과 `Content-type` / `Content-length` 를 보세요. 본문 내용보다 헤더가 틀린 경우가 꽤 많습니다.

브라우저가 멈춘 것 같으면, 서버가 헤더 끝 빈 줄을 안 보냈거나, 길이를 잘못 알려 줘서 서로 기다리는 경우를 의심해 볼 수 있습니다.

프록시에서는 응답 본문이 이미지 같은 바이너리일 수 있으므로, 줄 단위로만 다루려는 습관을 경계해야 합니다.

마무리 체크

3-line summary

- 응답의 핵심은 상태줄과 본문 길이/종류를 알려 주는 헤더다.
- Tiny는 보통 한 요청을 처리한 뒤 연결을 닫는 단순한 흐름을 보여 준다.
- curl -v 와 nc 는 브라우저보다 더 직접적으로 HTTP를 보여 주는 학습 도구다.

2 self-check questions

1) Content-type 이 잘못되면 브라우저는 어떤 식으로 혼란스러울까?
2) 응답이 오지 않을 때 왜 상태줄과 빈 줄부터 확인해야 할까?

초보자 메모

이 절을 “무엇을 읽고, 어떻게 처리하고, 무엇을 보내는가?”라는 한 문장으로 다시 말해 보면 이해가 훨씬 단단해집니다. 가능하면 브라우저 화면만 보지 말고, curl -v 또는 nc 로 이 절의 예시를 한 번 직접 확인해 보세요.

내 한 줄 메모

보강 노트 - 응답은 상태줄, 길이, 타입부터 본다

브라우저 화면만 보고 있으면 서버가 실제로 무슨 응답을 보냈는지 놓치기 쉽습니다. 초보자에게 가장 안전한 습관은 **상태줄**, **Content-type**, **Content-length** 를 먼저 보는 것입니다.

이 세 가지가 맞으면 본문을 이해하기가 쉬워지고, 셋 중 하나가 틀리면 본문이 멀쩡해 보여도 화면이 깨질 수 있습니다.

```
HTTP/1.0 200 OK
Server: Tiny Web Server
Content-length: 1234
Content-type: text/html

<html>...</html>
```

정적 응답에서는 본문 길이와 타입이 비교적 분명하게 들어간다.

응답 요소	초보자 해석	문제가 생기면 보이는 현상
HTTP/1.0 200 OK	요청이 성공했다는 선언	404, 403, 501이면 기대한 파일/메서드가 아니다
Content-type	이 본문을 어떤 종류로 볼지 알려 줌	HTML인데 text/plain 으로 보내면 화면이 다르게 보일 수 있다
Content-length	본문이 몇 바이트인지 알려 줌	길이가 틀리면 중간에서 끊기거나 더 기다릴 수 있다
연결 종료	HTTP/1.0 흐름에서는 메시지 끝 신호가 되기도 함	동적 응답에서 길이 헤더가 없으면 close 타이밍이 더 중요해진다

Tiny 동적 응답에서 눈여겨볼 점

Tiny의 동적 경로에서는 부모가 상태줄과 일부 헤더를 먼저 보내고, CGI 프로그램이 `Content-type` 과 본문을 이어서 출력합니다. 이 경우 `Content-length` 가 없을 수도 있고, 메시지 끝은 연결 종료가 알려 주기도 합니다.

마무리 체크

3-line summary

- 응답을 읽을 때는 먼저 성공/실패 상태와 본문 종류, 길이를 본다.
- Content-type 이 틀리면 본문이 맞아도 브라우저 해석이 어긋날 수 있다.
- 학습용 HTTP/1.0 흐름에서는 연결 종료 자체가 메시지 끝 신호가 되기도 한다.

2 self-check questions

1) 왜 동적 응답에서는 Content-length 가 항상 가장 먼저 눈에 띄지 않을 수 있을까?
2) 브라우저 화면보다 curl -v 가 디버깅에 더 좋은 이유는 무엇일까?

초보자 메모

이 절을 “무엇을 읽고, 어떻게 처리하고, 무엇을 보내는가?”라는 한 문장으로 다시 말해 보면 이해가 훨씬 단단해집니다. 가능하면 브라우저 화면만 보지 말고, curl -v 또는 nc 로 이 절의 예시를 한 번 직접 확인해 보세요.

내 한 줄 메모

CHAPTER 7 - 11.5.4 Serving Dynamic Content

7. CGI: 서버가 다른 프로그램을 실행해 결과를 보내는 법

CSAPP 11.5.4 Serving Dynamic Content에 대응

이 장의 학습 목표

- CGI가 별도 프로그램 실행 + 표준출력 연결이라는 구조임을 설명할 수 있다
- QUERY_STRING, fork, execve, dup2가 왜 필요한지 큰 흐름에서 말할 수 있다
- Tiny의 동적 콘텐츠 경로에서 부모와 자식 프로세스의 역할을 구분할 수 있다

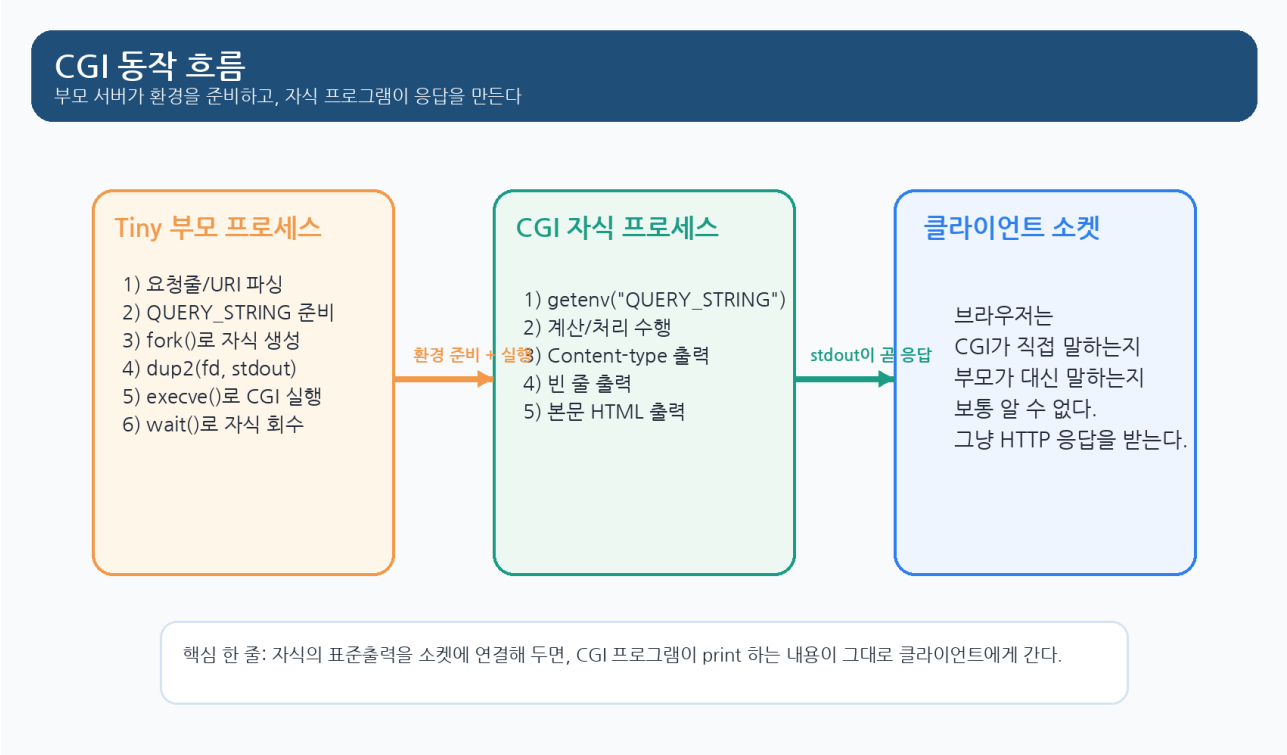
시각 자료 가이드

- CGI 흐름 그림: 브라우저 → Tiny 부모 → CGI 자식으로 이어지는 경로를 보여 주기 위해
- 짧은 부모/CGI 코드 예제: stdout 연결 감각을 글보다 빨리 잡게 하기 위해

왜 먼저 알아두면 좋은가

11.5.4는 많은 초보자가 가장 낯설다고 느끼는 부분이지만, 사실 핵심은 하나입니다. **부모 서버가 직접 문자열을 다 만들지 않고, 다른 프로그램의 표준출력을 소켓에 꽂아 준다**는 점입니다.

CGI(Common Gateway Interface)는 오래된 방식이지만, 웹 서버가 동적 콘텐츠를 어떻게 만들 수 있는지를 배우기에는 매우 좋은 교육용 모델입니다. 아이디어는 단순합니다. 서버가 요청을 분석한 뒤, 필요한 정보를 넘기고, 다른 프로그램을 실행시켜 그 프로그램의 출력을 클라이언트에게 보내는 것입니다.



부모 프로세스는 연결과 환경 설정을 맡고, CGI 자식 프로세스는 실제 출력(Content-type + 본문)을 만든다.

QUERY_STRING은 어디에 쓰이는가

GET 기반 동적 요청에서 **쿼리 문자열(query string)**은 보통 URL의 ? 뒤에 붙습니다. Tiny는 이것을 잘라 `QUERY_STRING` 환경 변수(environment variable, 프로그램 실행 전에 OS가 넘겨 주는 문자열 값)로 넣어 CGI 프로그램이 읽게 합니다.

```
if (Fork() == 0) {
    setenv("QUERY_STRING", cgiargs, 1);
    Dup2(fd, STDOUT_FILENO);
    Execve(filename, emptylist, environ);
}
Wait(NULL);
```

핵심은 자식의 stdout을 소켓으로 바꾸는 Dup2 이다. 그 뒤 execve 로 CGI 프로그램을 실행하면, CGI가 print 하는 내용이 곧 클라이언트로 간다.

fork 는 부모 프로세스를 복제해 자식 프로세스를 만드는 시스템 호출입니다. **execve** 는 현재 프로세스를 다른 실행 파일로 덮어씁니다. **dup2** 는 파일 디스크립터를 다른 번호에 복제하는데, 여기서는 자식의 **표준출력(stdout, 프로그램이 보통 화면으로 보내는 기본 출력 통로)**이 소켓을 가리키게 만들어 줍니다. 이 한 줄이 CGI의 핵심입니다.

CGI 프로그램은 왜 헤더도 스스로 출력해야 하는가

```
printf("Content-type: text/html\r\n\r\n");
printf("<p>adder 결과 = %d</p>\n", a + b);
```

Tiny 부모가 본문을 대신 만들어 주지 않기 때문에, CGI 프로그램은 최소한 Content-type 과 본문을 자기 손으로 출력해야 한다.

동적 경로에서는 부모가 이제부터 이 자식이 말한다고 무대를 넘겨주는 느낌으로 이해하면 좋습니다. 부모는 환경을 준비하고 실행을 시키고, 실제 본문 생성은 CGI 프로그램이 합니다. 그래서 CGI 프로그램이 헤더를 빼먹으면 브라우저도 해석하기 어려워집니다.

대표 CGI 환경 변수	초보자용 설명
QUERY_STRING	GET 요청의 인자 문자열
REQUEST_METHOD	GET 인지 POST 인지 같은 요청 방법
SERVER_PORT	부모 서버가 듣고 있는 포트
REMOTE_ADDR	클라이언트 IP 주소
CONTENT_LENGTH	POST 본문 길이(POST일 때)

GET vs POST를 여기서만 가볍게 정리

CSAPP는 CGI 설명에서 GET 요청이면 인자가 URI 안에 있고, POST 요청이면 본문에 있다는 점을 짚습니다. Tiny 예제는 GET 중심이므로, 초반에는 동적 요청의 인자가 `QUERY_STRING` 으로 전달된다는 감각만 확실히 잡아도 충분합니다.

POST 기반 CGI를 만들려면 표준입력(stdin)도 소켓으로 재지정해 본문을 읽게 하는 방식까지 생각해야 합니다.

자주 막히는 지점

CGI 파일에 실행 권한이 없으면 403 또는 실행 실패가 납니다.

자식이 `Content-type` 헤더를 출력하지 않으면 브라우저가 결과를 이상하게 보여 줄 수 있습니다.

부모가 본문을 직접 만드는 줄 오해하면 흐름이 헷갈립니다. 동적 경로에서는 표준출력을 소켓에 연결한다를 먼저 기억하세요.

마무리 체크

3-line summary

- CGI의 핵심은 서버가 별도 프로그램을 실행하고, 그 표준출력을 클라이언트 소켓으로 연결하는 것이다.
- QUERY_STRING 은 GET 요청의 인자를 CGI 프로그램에 전달하는 대표적인 통로다.
- 부모 Tiny와 CGI 자식은 역할이 다르며, 실제 본문은 자식이 출력한다.

2 self-check questions

1) 왜 dup2(fd, STDOUT_FILENO)가 CGI의 핵심 한 줄이라고 불릴까?
2) 동적 콘텐츠 경로에서 부모와 자식이 각각 맡는 책임은 무엇인가?

초보자 메모

이 절을 “무엇을 읽고, 어떻게 처리하고, 무엇을 보내는가?”라는 한 문장으로 다시 말해 보면 이해가 훨씬 단단해집니다. 가능하면 브라우저 화면만 보지 말고, curl -v 또는 nc 로 이 절의 예시를 한 번 직접 확인해 보세요.

내 한 줄 메모

보강 노트 - 부모/자식 프로세스와 파일 디스크립터 추적표

CGI가 어렵게 느껴지는 가장 큰 이유는 **프로세스가 둘**이 되고, **fd의 주인이 바뀌는 순간**이 있기 때문입니다. 이때는 코드를 줄 단위로 보지 말고, 순간마다 “지금 stdout 이 누구를 가리키는가?”만 따라가면 됩니다.

순간	부모 프로세스	자식 프로세스	핵심 포인트
fork 전	connfd를 들고 있음	아직 없음	서버가 클라이언트와 연결된 상태
fork 직후	connfd를 계속 들고 있음	connfd를 복사받음	부모와 자식 모두 같은 연결을 볼 수 있다
dup2 후	그대로	stdout이 connfd를 가리킴	printf 출력이 이제 네트워크로 간다
execve 후	wait로 자식 종료를 기다림	CGI 프로그램으로 바뀜	새 프로그램이지만 stdout 연결은 유지된다

```
child stdout ---> connfd ---> browser
printf("Content-type: text/html\r\n\r\n");
printf("<p>hello</p>\n");
```

CGI 프로그램이 표준출력에 쓰는 문자열이 곧 브라우저로 가는 응답 바이트가 된다.

이 장면을 한 문장으로 말하면

Tiny 부모는 “무대를 깔아 주는 사람”이고, CGI 자식은 “실제로 말하는 사람”입니다. 그래서 동적 경로에서는 부모보다 자식의 stdout 흐름을 더 눈여겨보는 편이 좋습니다.

마무리 체크

3-line summary

- CGI 이해의 핵심은 프로세스 둘과 stdout 연결 변화를 추적하는 것이다.
- dup2 이후에는 자식의 printf 출력이 파일이 아니라 네트워크로 간다.
- execve 로 프로그램이 바뀌어도 열린 fd 연결은 이어질 수 있다.

2 self-check questions

1) 왜 dup2 한 줄이 CGI를 이해하는 데 가장 중요한가?
2) 부모와 자식이 같은 connfd 를 본다는 사실은 어떤 의미를 가질까?

초보자 메모

이 절을 “무엇을 읽고, 어떻게 처리하고, 무엇을 보내는가?”라는 한 문장으로 다시 말해 보면 이해가 훨씬 단단해집니다. 가능하면 브라우저 화면만 보지 말고, curl -v 또는 nc 로 이 절의 예시를 한 번 직접 확인해 보세요.

내 한 줄 메모

CHAPTER 8 - 11.6 Tiny Web Server 개요

8. Tiny는 앞의 개념들을 어떻게 한 파일에 묶는가

CSAPP 11.6 Putting It Together: The Tiny Web Server에 대응

이 장의 학습 목표

- Tiny main 루프와 doit 함수가 어떤 관계인지 설명할 수 있다
- Tiny가 왜 GET만 지원하는지, 왜 반복적 서버인지 말할 수 있다
- Tiny의 주요 함수 이름을 보고 역할을 짐작할 수 있다

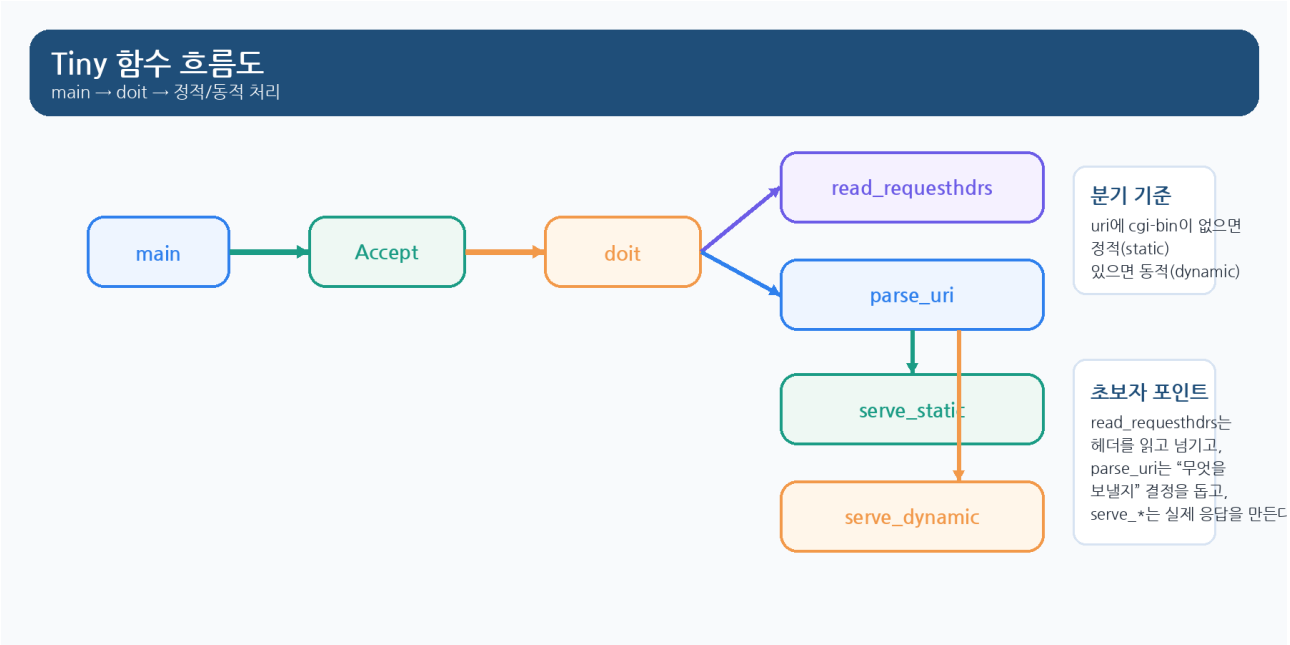
시각 자료 가이드

- Tiny 함수 흐름도: main → doit → serve_* 관계를 한눈에 보기 위해
- 함수 책임 표: 긴 코드가 아니라 역할 이름으로 길을 잃지 않게 하기 위해

왜 먼저 알아두면 좋은가

11.6은 드디어 앞에서 배운 모든 조각이 실제 코드로 만나는 구간입니다. 여기서 중요한 것은 코드를 줄 단위로 암기하는 것이 아니라, **각 함수가 어느 질문에 답하는가**를 보는 것입니다.

CSAPP가 Tiny를 왜 제시하는지부터 기억하면 좋습니다. Tiny는 기능이 많지는 않지만, **소켓 연결**, **HTTP 요청 해석**, **정적 파일 응답**, **CGI 실행**이라는 핵심 요소를 짧은 C 코드 안에 모두 담고 있습니다. 그래서 네트워크 프로그래밍, 시스템 호출, 파일 I/O, 프로세스 제어가 한 화면에서 만납니다.



Tiny를 읽을 때는 함수가 길게 느껴져도 결국 main → doit → parse_uri → serve_* 흐름으로 압축해 볼 수 있다.

Tiny의 가장 큰 뼈대

```
listenfd = Open_listenfd(argv[1]);
while (1) {
    connfd = Accept(listenfd, ...);
    doit(connfd);
    Close(connfd);
}
```

main 루프는 한 연결을 받아 doit 한 번 수행한 뒤 닫는 반복적 서버 구조다.

Tiny는 **반복적(iterative) 서버**이기 때문에, 한 요청을 처리하는 동안 다른 요청은 기다립니다. 현실의 고성능 웹 서버는 이렇게 만들지 않지만, 학습용으로는 흐름이 매우 선명합니다.

또한 Tiny는 **GET 메서드만 지원**합니다. 이것 역시 단점이 아니라 장점입니다. 초반에는 HTTP 전체를 다 담으려 하지 않고, 웹 서버의 가장 핵심적인 읽기-해석-응답 흐름에만 집중하게 해 주기 때문입니다.

함수	무슨 질문에 답하는가?
main	어떻게 포트를 열고 반복해서 연결을 받을까?
doit	이 한 요청을 어떻게 처리할까?
read_requesthdrs	헤더 끝 빈 줄까지 어떻게 읽을까?
parse_uri	이 URI는 정적인가 동적인가? 파일/인자는 무엇인가?
serve_static	파일을 어떤 헤더와 함께 어떻게 보낼까?
serve_dynamic	CGI를 어떤 환경에서 실행해 결과를 보낼까?
clienterror	실패했을 때 HTTP 오류 응답을 어떻게 보낼까?

코드를 읽는 순서 추천

처음부터 위에서 아래로 모든 줄을 따라가기보다, **main → doit → parse_uri → serve_static / serve_dynamic** 순서로 큰 가지를 먼저 잡으세요.

그 다음에 필요할 때 `read_requesthdrs`, `clienterror`, `get_filetype` 같은 보조 함수를 들어가 보는 편이 훨씬 덜 힘듭니다.

Tiny가 일부러 단순한 이유

실제 서버라면 동시성, 보안, 캐시, 더 많은 메서드, 더 다양한 헤더 처리, 더 정교한 오류 처리가 필요합니다. Tiny는 이 모든 현실 문제를 잠시 덜어 낸 덕분에 핵심 구조가 선명해집니다.

마무리 체크

3-line summary

- Tiny는 소켓, HTTP, 파일 I/O, 프로세스 제어를 한 파일에 묶은 학습용 서버다.
- main은 연결을 반복해서 받고, doit은 한 요청을 실제로 처리한다.
- Tiny의 단순함은 기능 부족이 아니라 핵심 구조를 드러내기 위한 설계다.

2 self-check questions

1) Tiny 코드를 읽을 때 함수 이름을 어떤 질문과 연결하면 좋은가?
2) 왜 Tiny가 GET만 지원하고도 학습 가치가 높은가?

초보자 메모

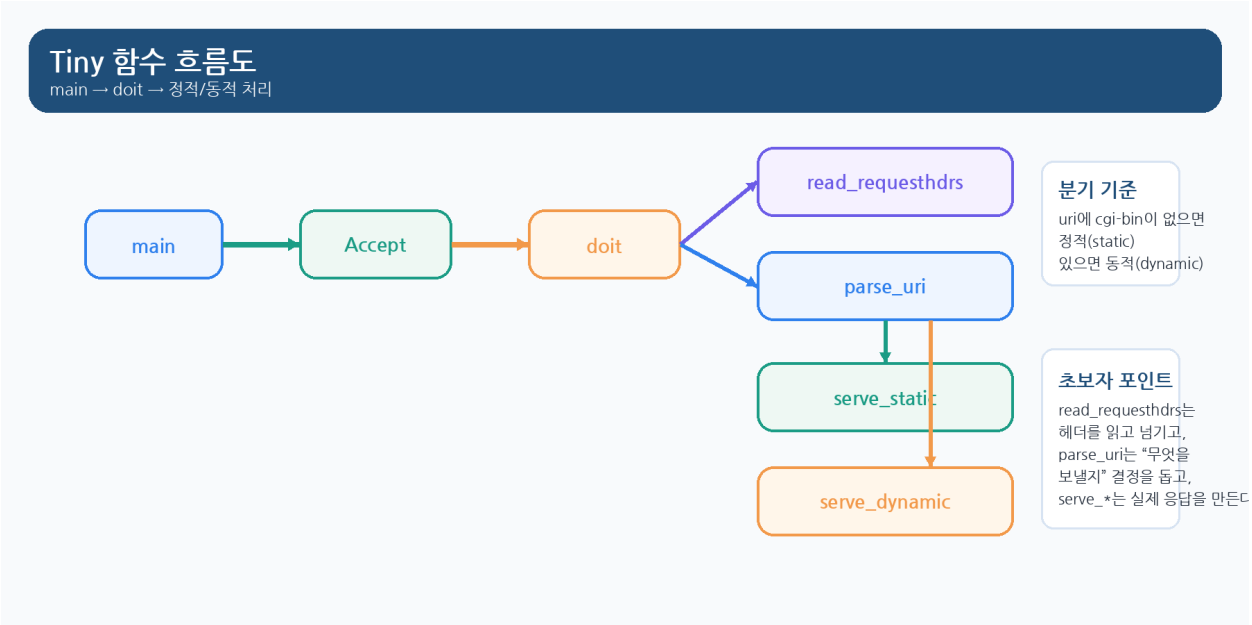
이 절을 “무엇을 읽고, 어떻게 처리하고, 무엇을 보내는가?”라는 한 문장으로 다시 말해 보면 이해가 훨씬 단단해집니다. 가능하면 브라우저 화면만 보지 말고, curl -v 또는 nc 로 이 절의 예시를 한 번 직접 확인해 보세요.

내 한 줄 메모

보강 노트 - Tiny 한 요청 전체를 끝까지 따라 걷기

코드를 함수별로 따로 보면 이해가 되다가도, 실제 요청 하나가 들어왔을 때 전체 흐름이 머릿속에서 이어지지 않는 경우가 많습니다. 그래서 **한 요청을 처음부터 끝까지 한 번에 추적**해 보는 연습이 중요합니다.

순서	무슨 일이 일어나는가	초보자 질문
1	main 이 listenfd 를 만들고 연결을 기다린다	지금은 새 손님을 기다리는 단계인가?
2	Accept 가 connfd 를 만든다	누구와 대화할 창구가 생겼는가?
3	doit 가 요청줄과 헤더를 읽는다	무엇을 달라는 요청인가?
4	parse_uri 가 정적/동적을 고른다	파일인가, 실행인가?
5	serve_static 또는 serve_dynamic 이 응답을 만든다	누가 실제 바이트를 쓰는가?
6	Close(connfd) 로 한 거래를 마무리한다	이번 요청은 여기서 끝났는가?



함수 이름을 외우기보다, 요청 하나가 어떤 분기와 검사를 거쳐 응답으로 바뀌는지만 따라가면 된다.

마무리 체크

3-line summary

- Tiny를 읽을 때는 함수 목록보다 요청 하나의 이동 경로를 따라가는 편이 쉽다.
- Accept, doit, parse_uri, serve_static/serve_dynamic, Close 가 큰 뼈대를 이룬다.
- 각 단계마다 지금은 기다림인지, 해석인지, 응답 생성인지 묻는 습관이 도움이 된다.

2 self-check questions

1) Tiny 한 요청 추적에서 가장 먼저 눈으로 따라가야 하는 fd 는 무엇일까?
2) parse_uri 가 Tiny 전체 흐름에서 유난히 중요한 이유는 무엇일까?

초보자 메모

이 절을 "무엇을 읽고, 어떻게 처리하고, 무엇을 보내는가?"라는 한 문장으로 다시 말해 보면 이해가 훨씬 단단해집니다. 가능하면 브라우저 화면만 보지 말고, curl -v 또는 nc 로 이 절의 예시를 한 번 직접 확인해 보세요.

내 한 줄 메모

CHAPTER 9 - Tiny 정적 경로 깊게 보기

9. read_requesthdrs, parse_uri, serve_static를 연결해서 읽기

11.6 Tiny의 요청 읽기와 정적 응답 경로를 해설

이 장의 학습 목표

- read_requesthdrs 가 왜 헤더를 읽고도 거의 사용하지 않는지 설명할 수 있다
- parse_uri 의 정적 경로 규칙과 기본 파일 확장 동작을 말할 수 있다
- serve_static 이 어떤 순서로 헤더와 본문을 보내는지 설명할 수 있다

시각 자료 가이드

- Tiny 함수 흐름도 재사용: 정적 가지를 강조하기 위해
- 정적 경로 체크리스트: 404/403/Content-type 과의 연결을 보이기 위해

왜 먼저 알아두면 좋은가

Tiny를 읽을 때 제일 자주 만나는 경로는 정적 파일 경로입니다. 그래서 정적 응답 흐름을 먼저 완전히 이해해 두면 나머지도 훨씬 쉬워집니다.

요청이 들어오면 Tiny의 `doit` 는 먼저 요청줄을 읽고 메서드, URI, 버전을 분리합니다. 메서드가 GET이 아니면 바로 `clienterror` 로 501 응답을 보내고 돌아갑니다.

```
Rio_readinitb(&rio, fd);
Rio_readlineb(&rio, buf, MAXLINE);
sscanf(buf, "%s %s %s", method, uri, version);
read_requesthdrs(&rio);
```

요청줄을 읽고, 그 뒤 헤더 끝 빈 줄까지 읽는 흐름이 Tiny의 출발점이다.

`read_requesthdrs` 는 이름 그대로 요청 헤더를 읽습니다. 하지만 Tiny는 대부분의 헤더 정보를 사용하지 않습니다. 초보자에게는 이 부분이 이상하게 보일 수 있는데, **헤더를 이용하지 않더라도 헤더 영역을 끝까지 소비해야** 다음 단계로 안전하게 넘어갈 수 있다고 이해하면 좋습니다.

parse_uri: Tiny만의 로컬 규칙

```
if (!strstr(uri, "cgi-bin")) {  // static
    strcpy(cgiargs, "");
    strcpy(filename, ".");
    strcat(filename, uri);
    if (uri[strlen(uri)-1] == '/')
        strcat(filename, "home.html");
    return 1;
}
```

정적 요청이면 현재 디렉터리 기준 상대 경로로 바꾸고, 슬래시로 끝나면 home.html 을 붙인다.

이 코드는 여러 중요한 사실을 알려 줍니다. 첫째, Tiny는 **정적 루트 디렉터리를 현재 디렉터리(.)**로 둡니다. 둘째, URI가 슬래시로 끝나면 기본 파일 이름으로 `home.html` 을 붙입니다. 셋째, `cgi-bin` 이 없으면 정적이라고 간주합니다.

즉 사용자가 `http://localhost:8000/` 만 입력해도 Tiny는 내부적으로 `./home.html` 을 찾아 보게 됩니다. 이 기본 파일 규칙을 이해하면 왜 루트 요청이 특정 파일로 연결되는지 자연스럽게 보입니다.

serve_static: 헤더를 쓰고, 파일을 복사한다

```
get_filetype(filename, filetype);
sprintf(buf, "HTTP/1.0 200 OK\r\n");
sprintf(buf, "%sContent-length: %d\r\n", buf, filesize);
sprintf(buf, "%sContent-type: %s\r\n\r\n", buf, filetype);
Rio_writen(fd, buf, strlen(buf));
```

정적 경로에서는 먼저 응답 헤더를 만들고, 그 뒤 파일 본문을 보낸다.

정적 경로에서는 먼저 `stat` 로 파일이 존재하는지, 정규 파일인지, 읽기 권한이 있는지 확인합니다. 여기서 실패하면 404 또는 403으로 이어집니다.

그 다음 `get_filetype` 이 파일 확장자를 보고 MIME 타입을 정하고, `Content-length` 와 `Content-type` 이 포함된 응답 헤더를 보냅니다. 마지막으로 파일 내용을 실제로 클라이언트 소켓으로 복사합니다.

CSAPP 원문 Tiny는 이 복사에 **mmap(memory map, 파일 내용을 가상 메모리 영역에 매핑해 메모리처럼 다루는 기법)** 을 사용합니다. 초보자는 mmap 세부 구현보다, 결국 파일 바이트를 소켓에 써서 보내는 것이라는 큰 흐름만 먼저 잡아도 충분합니다.

증상	정적 경로에서 먼저 확인할 것
404 Not Found	filename 으로 바뀐 실제 경로가 맞는가? 기본 home.html 이 존재하는가?
403 Forbidden	파일 읽기 권한이 있는가? 정규 파일인가?
브라우저가 이상하게 표시	Content-type 이 파일 확장자에 맞게 설정되었는가?
응답이 끊기거나 멈춤	Content-length 와 실제 파일 크기가 일치하는가?

정적 경로를 볼 때의 핵심 질문

이 URI는 최종적으로 어떤 로컬 파일명으로 바뀌는가?

그 파일은 존재하는가, 읽을 수 있는가, 크기는 얼마인가?

브라우저가 이것을 어떤 타입으로 해석하길 기대하는가?

마무리 체크

3-line summary

- read_requesthdrs 는 헤더를 잘 쓰기 위해서가 아니라, 헤더 끝까지 안전하게 지나가기 위해서도 중요하다.
- parse_uri 는 Tiny만의 로컬 규칙으로 URI를 파일명과 인자로 나눈다.
- serve_static 은 존재/권한 확인 → 헤더 작성 → 파일 바이트 전송 순서로 이해하면 된다.

2 self-check questions

1) 루트 경로 요청이 왜 home.html 과 연결될 수 있을까?
2) 정적 파일 응답에서 Content-type 과 Content-length 는 각각 어떤 일을 할까?

초보자 메모

이 절을 "무엇을 읽고, 어떻게 처리하고, 무엇을 보내는가?"라는 한 문장으로 다시 말해 보면 이해가 훨씬 단단해집니다. 가능하면 브라우저 화면만 보지 말고, curl -v 또는 nc 로 이 절의 예시를 한 번 직접 확인해 보세요.

내 한 줄 메모

보강 노트 - 정적 경로에서 404와 403은 어떻게 갈라질까

정적 경로는 상대적으로 단순하지만, 초보자가 가장 자주 만나는 오류도 여기서 많이 나옵니다. 같은 실패처럼 보여도 **파일이 없는 것**과 **권한이 없는 것**은 서버 입장에서 전혀 다른 상황입니다.

상황	Tiny가 확인하는 것	대표 결과
파일 경로가 없음	stat 가 실패하는가	404 Not Found
파일은 있으나 읽기 권한 없음	S_IRUSR 비트가 없는가	403 Forbidden
일반 파일이 아님	S_ISREG 검사가 실패하는가	오류 응답 또는 예상 밖 동작 방지
URI가 / 로 끝남	기본 파일 home.html 을 붙인다	디렉터리 요청을 대표 파일로 연결

```
curl -v http://localhost:8000/no-such-file.html
curl -v http://localhost:8000/secret.txt
```

같은 실패처럼 보여도 상태 코드는 404, 403처럼 달라질 수 있다.

디버깅 포인트

404면 **경로 해석**부터, 403이면 **권한과 파일 종류**부터 보는 편이 효율적입니다. 에러 페이지 HTML만 읽기보다, 왜 그 상태 코드가 선택되었는지를 먼저 생각해 보세요.

마무리 체크

3-line summary

- 정적 경로의 대표 오류는 경로 부재(404)와 권한 문제(403)다.
- Tiny는 stat 결과와 파일 모드를 보고 어떤 오류를 낼지 결정한다.
- 에러 페이지 본문보다 상태 코드가 먼저 원인 방향을 알려 준다.

2 self-check questions

1) 404와 403을 각각 어디서부터 추적하면 좋을까?
2) URI가 / 로 끝날 때 Tiny가 home.html 을 붙이는 이유는 무엇일까?

초보자 메모

이 절을 “무엇을 읽고, 어떻게 처리하고, 무엇을 보내는가?”라는 한 문장으로 다시 말해 보면 이해가 훨씬 단단해집니다. 가능하면 브라우저 화면만 보지 말고, curl -v 또는 nc 로 이 절의 예시를 한 번 직접 확인해 보세요.

내 한 줄 메모

CHAPTER 10 - Tiny 동적 경로 깊게 보기

10. serve_dynamic과 adder 예제로 CGI 전체 흐름 끝내기

11.6 Tiny의 동적 응답 경로와 adder 예제를 해설

이 장의 학습 목표

- parse_uri 의 동적 경로 분기와 cgiargs 추출을 설명할 수 있다
- serve_dynamic 의 부모/자식 역할 분담을 말할 수 있다
- adder 예제로 QUERY_STRING → HTML 응답 생성 흐름을 끝까지 따라갈 수 있다

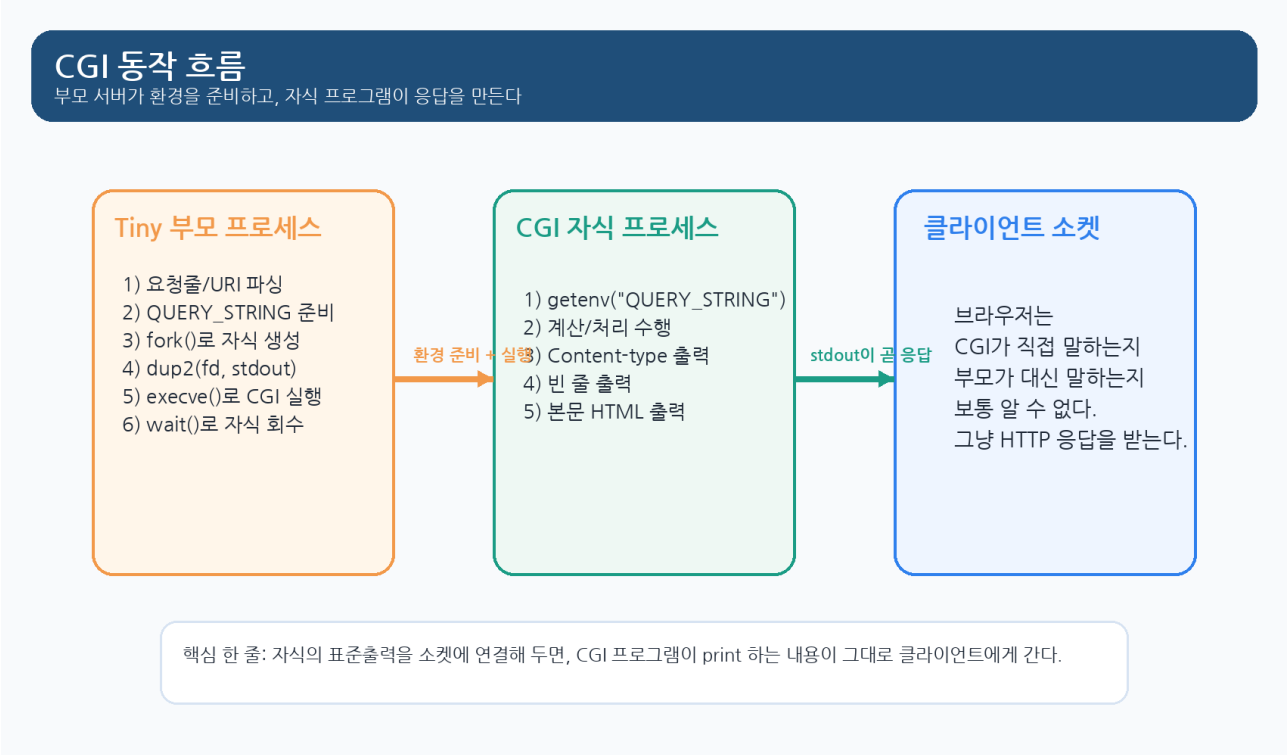
시각 자료 가이드

- CGI 흐름 그림 재사용: 동적 가지를 다시 시각적으로 고정하기 위해
- 실전 예제 타임라인: /cgi-bin/adder?15000&213 요청이 어떻게 처리되는지 보여 주기 위해

왜 먼저 알아두면 좋은가

동적 경로를 이해하면 웹 서버가 단순한 파일 배달 프로그램이 아니라, **프로그램 실행 결과를 웹으로 내보낼 수 있는 플랫폼**이라는 감각이 생깁니다.

동적 요청에서는 `parse_uri` 가 ? 뒤의 문자열을 잘라 `cgiargs` 에 넣고, 그 앞부분을 실행 파일 경로로 바꿉니다. 예를 들어 `/cgi-bin/adder?15000&213` 이 들어오면 실행 파일은 `./cgi-bin/adder`, 인자는 `15000&213` 으로 나뉩니다.



동적 경로에서는 부모 Tiny가 모든 HTML을 직접 만들지 않는다. 환경을 준비하고 CGI 자식이 말하도록 연결한다.

serve_dynamic의 핵심 줄

```
sprintf(buf, "HTTP/1.0 200 OK\r\n");
Rio_writen(fd, buf, strlen(buf));
sprintf(buf, "Server: Tiny Web Server\r\n");
Rio_writen(fd, buf, strlen(buf));

if (Fork() == 0) {
    setenv("QUERY_STRING", cgiargs, 1);
    Dup2(fd, STDOUT_FILENO);
    Execve(filename, emptylist, environ);
}
Wait(NULL);
```

부모는 응답의 앞부분을 열어 주고, 자식은 나머지 헤더와 본문을 stdout 으로 출력한다.

여기서 부모는 응답의 첫 부분을 먼저 보내지만, 나머지 구체적인 본문과 타입은 CGI 프로그램에 맡깁니다. 그래서 이 경로는 정적 경로보다 조금 덜 견고하지만, CGI의 본질을 보여 주기에는 매우 좋습니다.

`Wait(NULL)` 은 부모가 자식을 회수(reap)해 **좀비 프로세스(zombie process, 종료되었지만 부모가 아직 상태를 수거하지 않은 프로세스)**가 남지 않게 하는 역할을 합니다.

adder.c를 초보자 눈으로 보기

```
char *buf = getenv("QUERY_STRING");
p = strchr(buf, '&');
*p = '\0';
n1 = atoi(buf);
n2 = atoi(p+1);

printf("Content-type: text/html\r\n\r\n");
printf("<p>The answer is: %d + %d = %d</p>\n", n1, n2, n1+n2);
```

adder 는 QUERY_STRING 을 읽고, 문자열을 나눠 숫자로 바꾼 뒤, HTML 응답을 직접 출력한다.

이 짧은 CGI 프로그램 하나에 동적 웹의 본질이 들어 있습니다. 입력은 URL 뒤 쿼리 문자열에서 오고, 출력은 HTTP 헤더와 본문이 섞인 텍스트로 나갑니다. 즉 브라우저에서 보이는 계산 결과는 사실 **프로그램의 표준출력**입니다.

실습할 때는 브라우저보다 **curl -v** 로 먼저 확인해 보는 편이 좋습니다. 그러면 CGI가 보낸 `Content-type`, 빈 줄, HTML 본문이 순서대로 눈에 들어옵니다.

```
./tiny 8000
curl -v "http://localhost:8000/cgi-bin/adder?15000&213"
```

가장 간단한 동적 콘텐츠 테스트 예시.

단계	무슨 일이 일어나는가?
1. 요청 도착	Tiny가 요청줄에서 URI를 읽는다
2. 분기	cgi-bin 이 있으므로 동적 경로라고 판단한다
3. 인자 추출	? 뒤 문자열을 cgiargs / QUERY_STRING 으로 넘긴다
4. 자식 실행	fork 후 자식 stdout 을 소켓에 붙이고 execve 한다
5. CGI 출력	adder 가 Content-type 과 HTML 본문을 출력한다

단계	무슨 일이 일어나는가?
6. 응답 완료	부모가 자식을 기다리고 연결을 정리한다

자주 막히는 지점

CGI 파일에 실행 권한이 없으면 동적 경로가 403 또는 실행 실패로 이어집니다.

쿼리 문자열에 공백 같은 특수문자가 있으면 URL 인코딩이 필요할 수 있습니다. 초반에는 숫자/짧은 문자열로만 테스트하는 편이 좋습니다.

부모가 이미 일부 헤더를 보냈다는 점, 자식이 나머지를 보낸다는 점을 놓치면 응답이 어디서 만들어지는지 헷갈립니다.

마무리 체크

3-line summary

- 동적 경로에서는 URI가 실행 파일 경로와 쿼리 인자로 나뉜다.
- serve_dynamic 의 핵심은 QUERY_STRING 설정, stdout 재지정, execve 실행이다.
- adder 예제는 프로그램의 표준출력이 곧 웹 응답이 될 수 있음을 보여 준다.

2 self-check questions

1) 동적 경로에서 부모 Tiny와 adder 프로그램은 각각 무엇을 출력하는가?
2) 왜 CGI 파일의 실행 권한과 QUERY_STRING 처리가 자주 문제를 일으킬까?

초보자 메모

이 절을 “무엇을 읽고, 어떻게 처리하고, 무엇을 보내는가?”라는 한 문장으로 다시 말해 보면 이해가 훨씬 단단해집니다. 가능하면 브라우저 화면만 보지 말고, curl -v 또는 nc 로 이 절의 예시를 한 번 직접 확인해 보세요.

내 한 줄 메모

보강 노트 - adder 예제를 브라우저와 터미널 양쪽에서 실험하기

CGI는 눈으로만 보면 흐름이 모호할 수 있습니다. 하지만 같은 프로그램을 **브라우저**와 **터미널** 양쪽에서 호출해 보면, “입력은 URL에서 오고 출력은 stdout에서 간다”는 사실이 선명해집니다.

실험	관찰 포인트	의미
브라우저로 /cgi-bin/adder?1&2 열기	화면에 HTML 결과가 보임	stdout 결과가 브라우저에 렌더링됨
curl 로 같은 URL 요청	헤더와 본문을 텍스트로 볼 수 있음	브라우저가 숨긴 HTTP 층을 직접 확인
잘못된 인자 보내기	프로그램 출력이 어떻게 달라지는지 보기	동적 콘텐츠는 입력에 따라 응답이 바뀜
실행 권한 제거 후 요청	403/실행 실패 확인	동적 경로도 권한 영향을 크게 받음

```
curl -v 'http://localhost:8000/cgi-bin/adder?1&2'
chmod -x cgi-bin/adder
curl -v 'http://localhost:8000/cgi-bin/adder?1&2'
```

같은 CGI라도 권한, 인자, 헤더 출력 여부에 따라 결과가 크게 달라진다.

초보자에게 특히 좋은 관찰

adder 는 거대한 웹 프레임워크가 없어도 "웹 입력 → 프로그램 계산 → HTML 출력"이라는 전체 고리를 보여 줍니다. 그래서 작은 예제지만 Chapter 11의 핵심을 압축해서 담고 있습니다.

마무리 체크

3-line summary

- adder 예제는 URL 인자가 프로그램 입력이 되고, stdout 이 HTTP 본문이 되는 구조를 보여 준다.
- 같은 CGI를 브라우저와 curl 로 모두 실험하면 웹 층과 프로그램 층이 함께 보인다.
- 동적 콘텐츠는 입력과 실행 권한에 따라 결과가 쉽게 달라진다.

2 self-check questions

1) adder 가 Chapter 11의 핵심 축소판처럼 보이는 이유는 무엇일까?
2) 왜 CGI는 브라우저만으로 테스트하는 것보다 curl 도 함께 써 보는 편이 좋을까?

초보자 메모

이 절을 "무엇을 읽고, 어떻게 처리하고, 무엇을 보내는가?"라는 한 문장으로 다시 말해 보면 이해가 훨씬 단단해집니다. 가능하면 브라우저 화면만 보지 말고, curl -v 또는 nc 로 이 절의 예시를 한 번 직접 확인해 보세요.

내 한 줄 메모

CHAPTER 11 - Proxy Lab 관점으로 확장하기

11. 프록시는 왜 Tiny 다음 단계로 배우기 좋은가

Tiny에서 Proxy Lab으로 넘어갈 때의 사고법 정리

이 장의 학습 목표

- 프록시(proxy, 클라이언트와 원 서버 사이에서 요청/응답을 중계하는 프로그램)의 기본 역할을 설명할 수 있다
- 프록시가 서버 역할과 클라이언트 역할을 동시에 가진다는 점을 이해한다
- 브라우저 요청을 HTTP/1.0 형식으로 다시 만들어 보내는 이유를 말할 수 있다

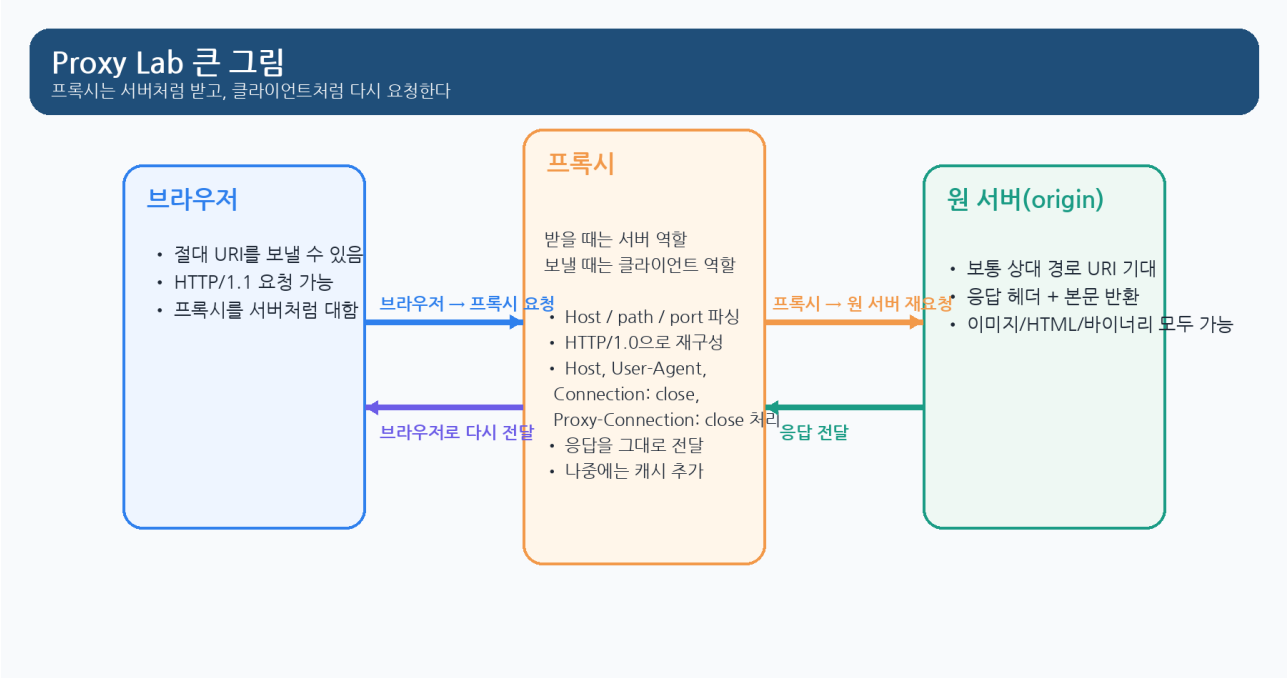
시각 자료 가이드

- Proxy 흐름 그림: 브라우저 ↔ 프록시 ↔ 원 서버 구조를 한 장면으로 보여 주기 위해
- Tiny vs Proxy 비교표: 재사용되는 감각과 새로 추가되는 책임을 나눠 보기 위해

왜 먼저 알아두면 좋은가

프록시는 Tiny와 별개의 주제가 아닙니다. Tiny가 한 서버의 입장에서 요청을 처리했다면, 프록시는 **중간자**의 입장에서 양쪽 말을 모두 이해하고 다시 전달해야 한다는 점만 다릅니다.

프록시(proxy)는 브라우저와 원 서버(origin server, 실제 콘텐츠를 가진 서버) 사이에서 요청과 응답을 중계하는 프로그램입니다. 브라우저는 프록시를 서버처럼 대하고, 프록시는 원 서버에게는 클라이언트처럼 행동합니다. 이 한 문장이 Proxy Lab 전체의 핵심입니다.



프록시는 받을 때는 서버, 보낼 때는 클라이언트가 된다. 그래서 Tiny에서 배운 서버 감각과 앞쪽 소켓 감각을 동시에 써야 한다.

브라우저는 프록시에게 종종 `GET http://host:port/path HTTP/1.1` 처럼 절대 URI가 들어간 요청을 보냅니다. 프록시는 여기서 호스트, 포트, 경로를 파싱한 뒤, 원 서버에게는 `GET /path HTTP/1.0` 같은 형태로 다시 보내야 합니다.

Proxy Lab류 과제에서는 보통 프록시가 항상 `Host`, `User-Agent`, `Connection: close`, `Proxy-Connection: close` 같은 헤더를 정리해 보내도록 요구합니다. 이유는 연결 관리와 실험 환경을 단순화하기 위해서입니다.

Tiny와 Proxy의 공통점과 차이

관점	Tiny	Proxy
받는 요청	브라우저가 보낸 요청을 자기 규칙으로 해석	브라우저 요청을 읽고 원 서버용 요청으로 재구성
응답 생성	정적 파일을 읽거나 CGI 실행 결과를 보냄	원 서버 응답을 받아 클라이언트로 다시 전달
역할 수	주로 서버 역할 하나	서버 역할 + 클라이언트 역할 둘 다
추가 고민	파일/권한/CGI	헤더 재작성, 바이너리 전달, 동시성, 캐시

Proxy에서 특히 어려운 이유

Proxy에서는 요청과 응답 둘 다 **그대로 흘려 보내는 능력**이 중요합니다. 특히 이미지처럼 바이너리 본문이 올 수 있으므로, 응답 본문을 텍스트 줄처럼 다루기만 하면 문제가 생길 수 있습니다.

또한 프록시는 한 번 읽은 요청을 다른 서버로 다시 쓰고, 그 서버가 보낸 응답을 다시 원래 클라이언트에게 써야 합니다. 즉 fd 가 두 개 이상 동시에 살아 있고, 지금 어느 소켓과 말하고 있는지 계속 의식해야 합니다.

Proxy Lab을 쉽게 느끼게 만드는 사고법

1) 먼저 브라우저 쪽에서는 Tiny처럼 **서버**라고 생각합니다. 요청줄과 헤더를 읽으세요.

2) 그다음 원 서버 쪽에서는 echo/Tiny 앞부분처럼 **클라이언트**라고 생각합니다. 연결하고 요청을 다시 쓰세요.

3) 마지막으로 서버 응답을 받아 클라이언트에게 그대로 흘려보냅니다. 이 단계는 파이프처럼 생각하면 편합니다.

동시성과 캐시는 왜 다음 장(혹은 과제 후반)에 붙는가

순차 프록시만으로도 이미 서버/클라이언트 이중 역할과 헤더 재작성이 어렵습니다. 그래서 과제는 보통 **순차 구현 → 동시성 → 캐시** 순서로 계단을 나눕니다.

동시성은 느린 클라이언트 하나가 전체를 막지 않게 하고, 캐시는 같은 객체를 매번 원 서버에 물어보지 않게 합니다.

마무리 체크

3-line summary

- 프록시는 브라우저에게는 서버, 원 서버에게는 클라이언트처럼 행동하는 중간자다.
- 브라우저 요청을 그대로 넘기지 않고, 원 서버가 이해하기 좋은 HTTP/1.0 스타일로 다시 만들 수 있다.
- Tiny에서 배운 요청 읽기/응답 쓰기 감각은 Proxy Lab에서도 그대로 재사용된다.

2 self-check questions

1) 프록시가 서버 역할과 클라이언트 역할을 동시에 가진다는 말은 구체적으로 무슨 뜻일까?
2) 왜 Proxy Lab은 Connection: close 같은 헤더를 일부러 넣게 할까?

초보자 메모

이 절을 “무엇을 읽고, 어떻게 처리하고, 무엇을 보내는가?”라는 한 문장으로 다시 말해 보면 이해가 훨씬 단단해집니다. 가능하면 브라우저 화면만 보지 말고, curl -v 또는 nc 로 이 절의 예시를 한 번 직접 확인해 보세요.

내 한 줄 메모

보강 노트 - Proxy Lab에서 자주 고쳐 쓰는 헤더들

Proxy는 받은 요청을 거의 그대로 전달하는 프로그램처럼 보이지만, 과제에서는 몇몇 헤더를 **의도적으로 다시 써야** 합니다. 이 과정에서 Proxy가 단순 중계기가 아니라 **HTTP를 이해하는 프로그램**이라는 사실이 드러납니다.

헤더	왜 만지는가	초보자 해석
Host	원 서버 기준으로 맞는 값을 유지해야 함	누구에게 보내는 요청인지 분명히 한다
User-Agent	과제에서 지정한 문자열로 통일하는 경우가 많음	클라이언트 정체성을 프록시가 다시 적는다
Connection: close	지속 연결 복잡도를 줄이기 위함	한 거래 후 끊는 단순 흐름 유지
Proxy-Connection: close	프록시와 클라이언트 사이 연결 정책을 명시	양쪽 연결을 헷갈리지 않게 한다

```
GET http://example.com:80/index.html HTTP/1.1
Host: example.com

# Proxy가 원 서버로 다시 보낼 때의 한 예
GET /index.html HTTP/1.0
Host: example.com
User-Agent: Mozilla/5.0 (...)
Connection: close
Proxy-Connection: close
```

브라우저가 보낸 요청과, 프록시가 원 서버로 정리해서 보내는 요청은 같아 보이면서도 일부가 달라진다.

머릿속 역할 전환

클라이언트와 대화할 때 Proxy는 서버처럼 행동하고, 원 서버와 대화할 때는 클라이언트처럼 행동합니다. Proxy Lab이 어려운 이유는 함수가 많아서가 아니라, **역할이 중간에 바뀌기 때문**입니다.

마무리 체크

3-line summary

- Proxy는 바이트를 그대로 복사하기보다, 필요에 따라 요청 헤더를 다시 구성한다.
- Connection/Proxy-Connection 을 close 로 두면 학습용 구현을 훨씬 단순하게 만들 수 있다.
- Proxy Lab의 핵심 어려움은 한 프로그램이 서버와 클라이언트 역할을 번갈아 맡는 데 있다.

2 self-check questions

1) 왜 Proxy는 Host 나 User-Agent 같은 헤더를 그냥 무시하면 안 될까?
2) Proxy가 서버처럼 보이다가도 클라이언트처럼 변하는 순간은 언제일까?

초보자 메모

이 절을 “무엇을 읽고, 어떻게 처리하고, 무엇을 보내는가?”라는 한 문장으로 다시 말해 보면 이해가 훨씬 단단해집니다. 가능하면 브라우저 화면만 보지 말고, curl -v 또는 nc 로 이 절의 예시를 한 번 직접 확인해 보세요.

내 한 줄 메모

CHAPTER 12 - 실습 및 테스트 체크리스트

12. echo, Tiny, Proxy를 어떤 순서로 테스트하면 좋은가

실습용 명령과 관찰 포인트를 정리한 테스트 챕터

이 장의 학습 목표

- echo → Tiny → Proxy 순서로 실습을 쌓는 이유를 설명할 수 있다
- curl, nc, 브라우저를 각각 언제 쓰면 좋은지 구분할 수 있다
- 정적/동적/프록시 테스트에서 무엇을 확인해야 하는지 체크리스트로 점검할 수 있다

시각 자료 가이드

- 실습 커버리지 매트릭스: 각 단계가 어떤 개념을 훈련하는지 보이기 위해
- 체크리스트 표: 실수 없이 한 단계씩 검증하기 위해

왜 먼저 알아두면 좋은가

실습은 많이 하는 것보다, **작은 단계를 확실히 통과하는 것**이 중요합니다. echo 감각이 약하면 Tiny가 어렵고, Tiny 감각이 약하면 Proxy가 갑자기 너무 커 보입니다.

실습 커버리지 매트릭스

echo → Tiny → Proxy로 갈수록 책임이 늘어난다

실습	연결 열기	요청 읽기	HTTP 해석	정적 파일	동적 CGI	헤더 재작성	중계/캐시
Echo	●	●	-	-	-	-	-
Tiny 정적	●	●	●	●	-	-	-
Tiny 동적	●	●	●	-	●	-	-
Proxy 순차	●	●	●	-	-	●	●
Proxy 동시성/캐시	●	●	●	-	-	●	●

한 단계씩 올라갈수록 "연결" 위에 "의미 해석"과 "역할 확장"이 추가된다. 그래서 echo 감각이 약하면 Tiny와 Proxy가 동시에 어렵게 느껴진다.

한 단계씩 올라갈수록 연결 위에 HTTP 해석과 역할 확장이 추가된다.

1단계 - echo 감각 확인

```
# 예시: echo 서버가 9000번 포트에서 대기
./echoserveri 9000
printf "hello\n" | nc localhost 9000
```

연결이 열리고, 보낸 바이트가 다시 오는지부터 확인한다.

체크 항목	확인 질문
연결	포트에 접속이 되는가?
왕복	보낸 바이트가 다시 오는가?
종료	프로그램이 끊김 없이 다음 연결을 다시 받을 수 있는가?

2단계 - Tiny 정적 콘텐츠 테스트

```
./tiny 8000
curl -v http://localhost:8000/
curl -I http://localhost:8000/
```

처음에는 브라우저보다 curl -v 로 상태줄과 헤더를 확인하는 편이 좋다.

☐ HTTP/1.0 200 OK 또는 기대한 오류 코드가 보인다	☐ 루트 요청(/)이 기본 파일로 연결된다
☐ Content-type 이 파일 종류와 맞다	☐ 존재하지 않는 파일은 404가 난다
☐ Content-length 값이 있다	☐ 권한이 없는 파일은 403 또는 접근 실패가 난다

3단계 - Tiny 동적 콘텐츠 테스트

```
./tiny 8000
curl -v "http://localhost:8000/cgi-bin/adder?1&2"
```

CGI는 브라우저 결과보다 헤더와 빈 줄이 먼저 맞는지 보는 것이 중요하다.

☐ 요청 URI에 cgi-bin 이 포함된다	☐ Content-type: text/html 이 출력된다
☐ QUERY_STRING 으로 인자가 전달된다	☐ 빈 줄 뒤에 HTML 본문이 온다
☐ CGI 파일에 실행 권한이 있다	☐ 부모가 자식을 wait 해 좀비가 쌓이지 않는다

4단계 - Proxy 기본 동작 테스트

```
# 예시: 프록시가 15213번 포트에서 대기
./proxy 15213

# curl에서 프록시 사용
curl -v --proxy http://localhost:15213 http://example.com/
```

브라우저 대신 curl --proxy 로 먼저 원 서버 요청/응답이 흘러가는지 확인한다.

☐ 브라우저/클라이언트 요청을 프록시가 읽는다	☐ HTTP/1.0 요청줄로 다시 쓴다
☐ Host, path, port 를 제대로 파싱한다	☐ 응답 헤더와 본문을 클라이언트로 다시 전달한다
☐ 원 서버에 연결이 된다	☐ 이미지/바이너리도 깨지지 않게 전달한다

30분 실습 루틴 예시

시간대	추천 행동	왜 이 순서가 좋은가
0-10분	echo 또는 nc 로 연결 왕복 확인	문제가 포트/연결인지 HTTP인지 먼저 분리할 수 있다
10-20분	Tiny 정적 + Tiny 동적을 curl -v 로 확인	파일/CGI 두 경로를 따로 검증할 수 있다
20-30분	Proxy 기본 전달과 헤더 재구성 확인	이미 검증된 Tiny 감각 위에 프록시 역할만 얹어 볼 수 있다

마무리 체크

3-line summary

- 실습은 echo → Tiny 정적 → Tiny 동적 → Proxy 순서로 쌓을 때 가장 안정적이다.
- curl -v 는 상태줄과 헤더를 보는 데, nc 는 손수 요청 형식을 시험하는 데 좋다.
- 각 단계에서 무엇이 통과되어야 다음 단계로 넘어가도 되는지 체크리스트로 관리하면 막힘이 줄어든다.

2 self-check questions

1) 왜 브라우저만으로 테스트하면 초반 디버깅이 오히려 어려워질 수 있을까?
2) Proxy 테스트 전에 Tiny 정적/동적 흐름을 먼저 확인하는 이유는 무엇일까?

초보자 메모

이 절을 “무엇을 읽고, 어떻게 처리하고, 무엇을 보내는가?”라는 한 문장으로 다시 말해 보면 이해가 훨씬 단단해집니다. 가능하면 브라우저 화면만 보지 말고, curl -v 또는 nc 로 이 절의 예시를 한 번 직접 확인해 보세요.

내 한 줄 메모

CHAPTER 13 - 흔한 함정과 디버깅 순서

13. 멈춤, 404, 403, 깨진 응답을 보면 어디부터 볼까

Tiny/Proxy를 디버깅할 때 자주 만나는 증상과 점검 순서

이 장의 학습 목표

- 증상별로 가장 먼저 의심할 지점을 정리할 수 있다
- 포트, 요청줄, 빈 줄, 권한, 헤더를 우선순위 있게 점검할 수 있다
- 반복적 서버와 동시성 서버의 병목 차이를 설명할 수 있다

시각 자료 가이드

- 디버깅 사다리 그림: 확인 순서를 시각적으로 고정하기 위해
- 반복적 vs 동시성 그림: 느린 클라이언트가 왜 전체를 막는지 보여 주기 위해

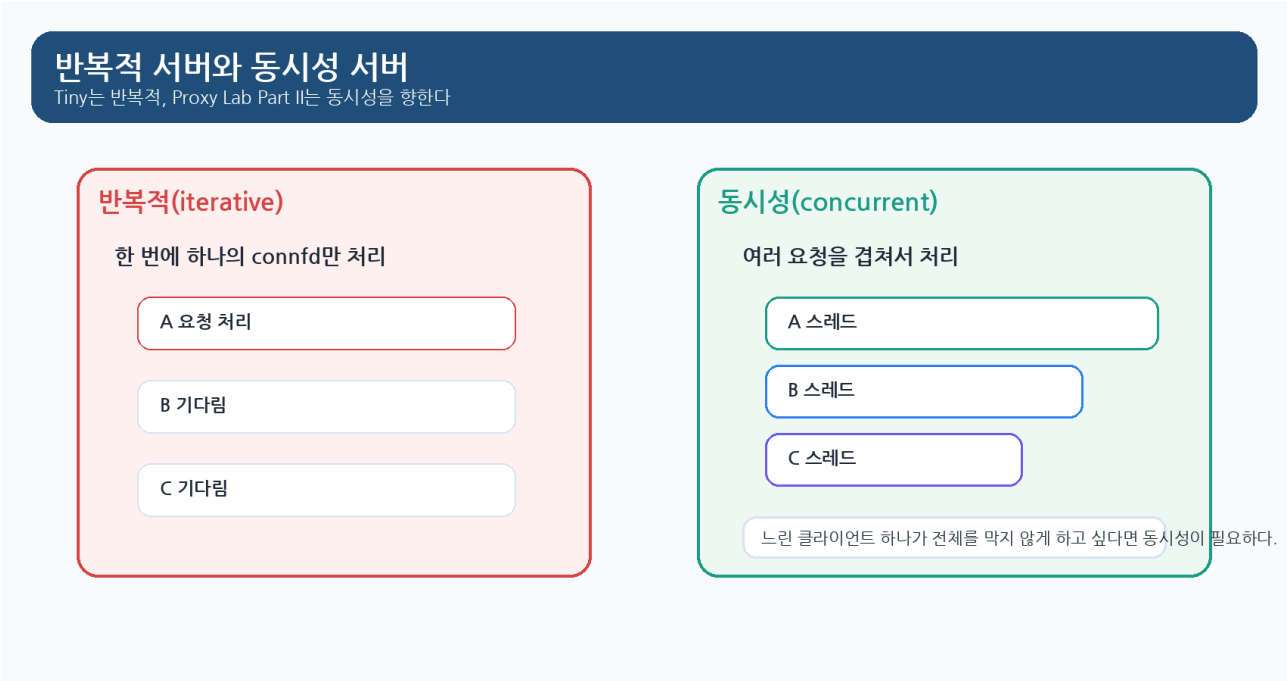
왜 먼저 알아두면 좋은가

디버깅은 많이 아는 것보다 **순서가 있는 것**이 중요합니다. 증상마다 랜덤으로 건드리기보다, 아래에서 위로 또는 위에서 아래로 일정한 사다리를 타면 훨씬 빨리 해결됩니다.

작동하지 않을 때는 포트와 프로세스부터, 그다음 요청/응답 형식, 마지막으로 권한과 길이를 본다.

증상	자주 있는 원인	첫 번째 확인
Connection refused	프로세스가 안 떠 있거나 포트가 다름	프로그램 실행 여부와 포트 번호
브라우저가 계속 기다림	빈 줄 누락, 길이 문제, 서버/클라이언트가 서로 기다림	요청/응답 헤더 끝과 Content-length

증상	자주 있는 원인	첫 번째 확인
404 Not Found	경로 변환 실패, 기본 파일 없음	parse_uri 결과와 실제 파일 위치
403 Forbidden	읽기/실행 권한 부족	파일 권한 비트와 정규 파일 여부
이미지/본문 깨짐	Content-type 또는 바이너리 전달 방식 문제	응답 헤더와 본문 복사 코드
한 클라이언트가 느리면 전체가 느림	반복적 서버 구조	반복적/동시성 여부



Tiny는 반복적 서버라 한 번에 하나를 처리한다. Proxy Lab 후반은 동시성으로 이 병목을 줄이는 방향으로 간다.

초보자용 점검 루틴

```
# 포트/프로세스 확인
ss -ltnp | grep 8000

# 요청/응답 확인
curl -v http://localhost:8000/
printf "GET / HTTP/1.0\r\n\r\n" | nc localhost 8000
```

도구는 적게, 순서는 고정해서 쓰는 편이 좋다.

디버깅 순서를 고정하면 좋은 이유

1) 프로세스/포트 문제는 코드가 맞아도 실패합니다. 그래서 제일 먼저 봅니다.

2) 그다음 요청줄/헤더 끝 빈 줄 같은 형식 문제를 봅니다. 이 단계에서 많은 멈춤이 해결됩니다.

3) 마지막으로 파일 경로, 권한, Content-type, 길이 같은 의미 문제를 봅니다.

Proxy에서 추가로 기억할 것

프록시는 **두 개의 연결**을 다루므로, 지금 로그가 클라이언트 쪽인지 원 서버 쪽인지 꼭 구분해 적어 두면 좋습니다.

응답 본문은 텍스트라고 가정하지 마세요. 이미지나 압축 데이터일 수도 있으므로 바이트 그대로 전달하는 습관이 중요합니다.

증상별 한 줄 처방

증상	바로 해 볼 명령/행동
연결 거절	프로세스를 다시 띄우고 포트 번호가 맞는지 확인
계속 기다림	curl -v 와 nc 로 빈 줄/길이/close 여부 확인
404	요청 URI 와 실제 파일 위치를 나란히 써 보기
403	ls -l 로 읽기/실행 권한 확인
이미지 깨짐	바이트 그대로 전달하는지, 줄 단위 함수만 쓰지 않았는지 확인

마무리 체크

3-line summary

- 디버깅은 프로세스/포트 → 형식(요청줄/빈 줄) → 의미(경로/권한/길이) 순서로 보면 효율적이다.
- 404와 403은 비슷해 보여도 각각 경로와 권한 문제를 먼저 의심하는 편이 좋다.
- 반복적 서버의 병목은 구조적인 것이며, 동시성으로 방향을 바꿔야 해결된다.

2 self-check questions

1) 브라우저가 계속 기다릴 때 가장 먼저 확인할 형식 요소는 무엇일까?
2) 반복적 서버와 동시성 서버의 차이는 실제 체감에서 어떻게 드러날까?

초보자 메모

이 절을 “무엇을 읽고, 어떻게 처리하고, 무엇을 보내는가?”라는 한 문장으로 다시 말해 보면 이해가 훨씬 단단해집니다. 가능하면 브라우저 화면만 보지 말고, curl -v 또는 nc 로 이 절의 예시를 한 번 직접 확인해 보세요.

내 한 줄 메모

CHAPTER 14 - 용어 사전과 원문 읽기 로드맵

14. Glossary + 이제 원문 Chapter 11을 어떻게 읽을까

마지막 정리: 용어를 한 줄씩 다시 붙잡고, 원문으로 들어갈 순서를 정리

이 장의 학습 목표

- 핵심 용어를 한 줄 정의로 다시 말할 수 있다
- 원문 Chapter 11을 어디를 빨리, 어디를 천천히 읽을지 계획할 수 있다
- Tiny와 Proxy Lab을 어떤 질문으로 다시 읽을지 정리할 수 있다

시각 자료 가이드

- 용어 표: 첫 읽기에서 자주 섞이는 단어를 빠르게 재정리하기 위해
- 읽기 로드맵 박스: 원문으로 넘어갈 때 길을 잃지 않게 하기 위해

왜 먼저 알아두면 좋은가

마지막 장의 목표는 더 많은 정보를 주는 것이 아니라, **지금 손에 남아야 할 단어와 읽는 순서**를 다시 정리하는 것입니다.

용어	한 줄 정의
클라이언트	요청을 시작하는 쪽 프로그램
서버	요청을 받아 응답을 보내는 쪽 프로그램
호스트	네트워크에 연결된 컴퓨터
포트	한 호스트 안에서 프로그램을 구분하는 번호
소켓	프로그램이 네트워크 연결을 다루는 통로
listenfd	새 연결을 기다리는 소켓 fd
connfd	받아들인 한 클라이언트와 실제로 대화하는 소켓 fd
HTTP	웹에서 요청과 응답 형식을 정한 규약
요청줄	메서드, URI, 버전이 들어 있는 요청 첫 줄
헤더	부가 정보를 담는 key: value 형태의 줄
빈 줄	헤더가 끝났음을 알리는 줄
상태줄	버전, 상태 코드, 상태 메시지가 들어 있는 응답 첫 줄
MIME 타입	본문 데이터 종류를 알려 주는 문자열
정적 콘텐츠	이미 있는 파일을 읽어 보내는 응답
동적 콘텐츠	프로그램을 실행해 결과를 만들어 보내는 응답

용어	한 줄 정의
CGI	서버가 외부 프로그램을 실행해 동적 응답을 만드는 방식
QUERY_STRING	GET 요청 인자를 담아 CGI에 넘기는 환경 변수
stdout	프로그램의 기본 출력 통로
proxy	클라이언트와 서버 사이에서 요청/응답을 중계하는 프로그램
origin server	실제 콘텐츠를 가진 원 서버
cache	다음 요청을 더 빨리 처리하려고 저장해 둔 응답 객체
iterative server	한 번에 하나의 요청만 처리하는 서버
concurrent server	여러 요청을 겹쳐 처리하는 서버

이제 원문 Chapter 11을 읽는 순서

1) 11.1~11.4는 함수 이름보다 그림과 예제 흐름 위주로 빠르게 예열합니다.

2) 11.5는 가장 천천히 읽습니다. 특히 11.5.2~11.5.4를 코드와 연결해 봅니다.

3) 11.6 Tiny는 함수 이름 옆에 역할을 메모하며 읽습니다.

4) 읽다가 막히면 이 문서의 URL/HTTP/CGI/Tiny 흐름 그림으로 다시 돌아옵니다.

원문을 읽을 때 계속 붙잡을 질문

이 줄은 요청을 읽는 줄인가, 응답을 쓰는 줄인가?

이 함수는 파일을 다루는가, 프로세스를 다루는가, 소켓을 다루는가?

지금 URI는 파일 경로로 바뀌는가, 프로그램 실행 경로로 바뀌는가?

이 동작은 Tiny 안에서 끝나는가, 아니면 Proxy처럼 다른 서버까지 이어지는가?

마지막 격려 메모

처음에는 11.5와 11.6이 낯설 수 있지만, 결국은 **연결을 맺고 - 요청을 읽고 - 규칙대로 해석하고 - 응답을 보내는 프로그램**이라는 한 문장으로 다시 묶입니다.

Tiny를 이해한 뒤 Proxy Lab을 보면 갑자기 커 보이던 과제가, 사실은 같은 흐름에 역할 하나가 더 붙은 문제라는 점이 보이기 시작할 것입니다.

마무리 체크

3-line summary

- 핵심 용어를 한 줄로 다시 말할 수 있으면 원문을 읽을 때 불안이 크게 줄어든다.
- 원문은 11.5를 가장 천천히, 11.6은 함수 역할 중심으로 읽으면 좋다.
- Tiny와 Proxy를 연결하는 질문을 잡아 두면 이후 실습도 훨씬 덜 흔들린다.

2 self-check questions

1) 원문 Chapter 11을 읽을 때 가장 천천히 읽어야 할 절은 어디일까?
2) 지금 내가 Tiny와 Proxy를 잇는 가장 중요한 한 문장을 어떻게 만들 수 있을까?

초보자 메모

이 절을 “무엇을 읽고, 어떻게 처리하고, 무엇을 보내는가?”라는 한 문장으로 다시 말해 보면 이해가 훨씬 단단해집니다. 가능하면 브라우저 화면만 보지 말고, curl -v 또는 nc 로 이 절의 예시를 한 번 직접 확인해 보세요.

내 한 줄 메모